

UNIVERSIDAD AUTÓNOMA DE SAN LUIS POTOSÍ



FACULTAD DE INGENIERÍA
ÁREA DE CIENCIAS DE LA COMPUTACIÓN

MARTINEZ SANCHEZ MAYRA ALIN
RODRIGUEZ DE LA TORRE AARON EMMANUEL
BRAGA DE LIMA BRENDA

INTERFACES GRÁFICAS CON APLICACIONES

M.I. Nuñez Varela Alberto Salvador

Reporte proyecto UNO

09/11/2026

SEMESTRE
2026-2027/I

Índice

1.Descripción general del proyecto

2.Arquitectura del proyecto

- 2.1 Capa de presentación (Windows Forms)
- 2.2 Capa de lógica del juego
- 2.3 Capa de acceso a datos (DAO)
- 2.4 Base de datos
- 2.5 API

3. Tabla de cuentas de GitHub

4. División de tareas

- 4.1 Braga de Lima Brenda
- 4.2 Martínez Sánchez Mayra Alin
- 4.3 Rodríguez de la Torre Aarón Emmanuel

5. Diagrama entidad-relación de la base de datos

6. Listado de commits

7. Cronograma de actividades

8. Resumen y gráfica de commits

9. Listado de ramas generadas

10. Capturas de la aplicación

11. Problemas encontrados y cómo los solucionaron

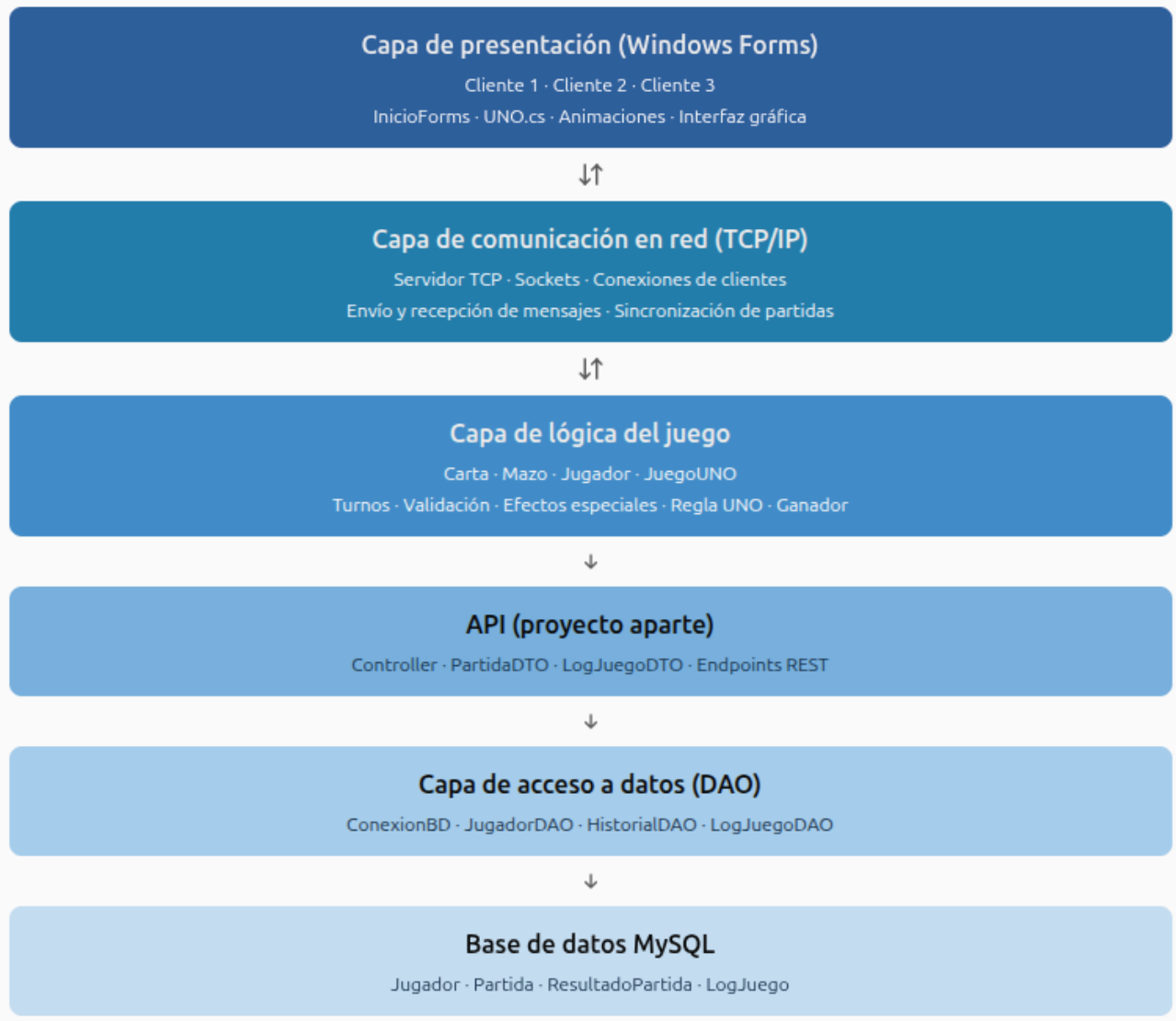
12. Conclusión personal

- Descripción general del proyecto

Este proyecto está enfocado en el desarrollo de un juego UNO gráfico en C#, con una arquitectura cliente-servidor que permite que tres jugadores participen simultáneamente desde computadoras diferentes conectadas a la misma red. La comunicación entre los clientes y el servidor se realiza mediante sockets TCP, mientras que la integración con una API y una base de datos MySQL permite almacenar y consultar la información de los jugadores, el historial de partidas ganadas y perdidas, y el registro detallado de cada movimiento. Asimismo, incluye un reporte técnico con la descripción del sistema, la división de tareas, el diagrama entidad-relación, evidencias de Git/GitHub (ramas y commits) y la descripción de los problemas encontrados y sus respectivas soluciones.

- Arquitectura del proyecto

El proyecto sigue una arquitectura en capas, desarrollada en C# con Windows Forms y conectada a una base de datos MySQL y a una API. Cada capa tiene una responsabilidad distinta.



1. Capa de presentación (Windows Forms)

Contiene el formulario de inicio (InicioForms) y el formulario principal del juego (UNO.cs). Muestra a los tres jugadores, sus cartas como botones, la carta actual, el turno, el botón UNO y el mensaje del ganador. Incluye las clases de animación (desplazamiento de cartas, abanico de cartas), la clase Utilidades y el uso de DoubleBuffered para evitar el parpadeo.

2. Capa de comunicación en red (TCP/IP)

Permite que tres jugadores participen en una misma partida desde computadoras diferentes conectadas a la red. Utiliza una arquitectura cliente-servidor mediante sockets TCP, donde el servidor administra las conexiones de los jugadores, recibe sus acciones y envía las respuestas correspondientes. La comunicación se realiza mediante mensajes JSON, que permiten transmitir información sobre el registro de jugadores, las jugadas, los turnos y el estado de la partida, manteniendo sincronizada la información entre los clientes.

3. Capa de lógica del juego

Aquí están las reglas de UNO:

- Carta: color, valor y tipo.
- Mazo: barajado, robo y pila de descarte.
- Jugador: ID, nombre y mano de cartas.
- JuegoUNO: estado de la partida, jugador actual, dirección, turnos, validación de cartas, cartas especiales (Reversa, Salta, +2, Comodín y +4), regla UNO con su penalización, detección del ganador y lista de perdedores.

4. Capa de acceso a datos (DAO)

- ConexionBD: conexión con MySQL mediante el paquete MySql.Data y métodos base para ejecutar consultas.
- JugadorDAO: consulta, creación y administración de jugadores, así como sus estadísticas.
- HistorialDAO: guardado de partidas, ganadores, participantes y resultados.
- LogJuegoDAO: registro de cada movimiento (cartas jugadas y robadas, cambios de color, turnos, acciones especiales y fecha/hora).

5. Base de datos

Tiene las tablas Jugador, Partida, ResultadoPartida y LogJuego, con sus llaves primarias, foráneas y relaciones. Almacena la información de los jugadores, las partidas realizadas, sus resultados y el historial de acciones.

6. API

Es un proyecto independiente que funciona como intermediario entre el servidor del juego y la capa de acceso a datos. Contiene los controladores, los DTO (PartidaDTO, LogJuegoDTO y otros objetos de transferencia de datos) y los endpoints REST necesarios para registrar partidas, participantes, jugadas, turnos, cartas robadas, acciones especiales y resultados finales. El

servidor TCP realiza solicitudes a la API para guardar la información en MySQL. El registro de eventos se realiza mediante operaciones asíncronas para evitar bloquear el procesamiento de la partida.

Flujo general: Los tres jugadores se conectan desde computadoras diferentes mediante la comunicación TCP/IP. Cada cliente envía sus acciones al servidor, que las procesa utilizando la lógica del juego y devuelve las respuestas correspondientes. A su vez, el servidor se comunica con la API para registrar las partidas y sus movimientos. La API utiliza los DAO para acceder a la base de datos MySQL, donde se almacena el historial completo de las partidas, incluyendo los participantes, las jugadas, las acciones especiales y los resultados finales.

- Tabla de cuentas de GitHub

INTEGRANTE	CUENTA DE GITHUB
Martinez Sanchez Mayra Alin	 alinsnhz Alin Sanchez
Rodriguez de la Torre Aaron Emmanuel	 Emmanuel1514
Braga de Lima Brenda	 brendabragaaaa

- División de tareas

BRAGA DE LIMA BRENDA

implementar clase Carta

Crear la clase Carta que representa cada carta del UNO. Debe contener la información necesaria para identificar su color, valor y tipo de carta.

Tareas:
Crear Carta.cs
Agregar propiedad de color.
Agregar propiedad de valor.
Agregar propiedad de tipo.
Crear constructor.
Probar creación de cartas.

implementar cartas del UNO

Implementar la creación de las cartas que forman el mazo del UNO.

Tareas:
Crear cartas de números 0–9.
Crear cartas Reversa.
Crear cartas Salta.
Crear cartas +2.
Crear Comodines.
Crear Comodines +4.
Verificar que el mazo tenga la cantidad correcta de cartas.

implementar clase Mazo

Crear la clase encargada de administrar el mazo de cartas.

Tareas:

Crear Mazo.cs.
Crear lista de cartas.
Crear método para generar el mazo.
Crear método para barajar.
Crear método para sacar/robar una carta.
Controlar las cartas restantes.

Repartir cartas iniciales

Implementar el reparto inicial de cartas a los tres jugadores.

Tareas:
Repartir 7 cartas a cada jugador.
Verificar que no se repartan cartas inexistentes.
Verificar cantidad de cartas restantes.
Probar reparto para 3 jugadores.

Administrar pila de descarte

Implementar la pila donde se colocan las cartas jugadas y controlar la carta que se encuentra actualmente en juego.

Tareas:
Crear pila/lista de descarte.
Agregar cartas jugadas.
Obtener carta superior.
Controlar qué sucede cuando se termina el mazo.

Completar los endpoints de partidas, eventos y resultados

Revisar los endpoints actuales de PartidaController.cs y los DTOs para que el servidor pueda registrar las acciones y el resultado final de una partida.

Tareas:
Revisar el endpoint de inicio de partida.
Revisar el registro de eventos y jugadas.

Agregar api/Partida/log-robar si se mantiene esa llamada.
Registrar las cartas jugadas y robadas cuando corresponda.
Registrar las acciones especiales y las penalizaciones.
Registrar al ganador y el resultado final.
Validar los DTOs y las respuestas HTTP.
Evitar registrar dos veces la misma acción.
Mantener la compatibilidad con los DAO actuales.

Centralizar el acceso a MySQL y eliminar escrituras desde los clientes

Asegurar que la API sea la encargada de escribir el historial y los resultados en MySQL, evitando conexiones independientes desde cada Windows Forms.

Tareas:
Revisar las operaciones de lectura y escritura de los DAO.
Conservar los DAO en el proyecto de la API.
Eliminar del flujo del cliente las escrituras directas en MySQL.
Modificar el cliente para consultar la API cuando necesite historial.
Registrar los movimientos desde el servidor a través de la API.
Revisar las claves foráneas y la relación entre partida, jugadores y movimientos.
Comprobar que una partida solo se finalice y registre una vez.

Integrar el servidor TCP con la API y MySQL

Conectar la lógica de la partida con el registro persistente, para que cada partida y su resultado queden almacenados correctamente.

Tareas:
Registrar la partida al comenzar.
Relacionar cada jugador con su participación.
Registrar los movimientos aceptados.
Registrar robos y efectos especiales.
Registrar el ganador al terminar.
Manejar errores HTTP sin duplicar operaciones.
Verificar los datos guardados en MySQL.
Comprobar que el servidor utiliza el identificador correcto de la partida.

implementar el juego completo en tres computadoras independientes

Realizar pruebas de integración con tres clientes Windows Forms conectados al mismo servidor TCP y con la API y MySQL activos.

Tareas:
Conectar tres computadoras a la misma red.
Iniciar el servidor TCP.
Iniciar la API y comprobar la conexión con MySQL.
Conectar los tres clientes.
Verificar el reparto inicial de cartas.
Confirmar que cada jugador solo ve su propia mano.
Comprobar los turnos y las reglas especiales.
Probar una desconexión inesperada.
Terminar una partida y revisar los registros en MySQL.
Corregir los errores detectados y guardar evidencia de las pruebas.

documentar la instalación, configuración y ejecución del UNO en red
Actualizar la documentación del proyecto para que el profesor pueda ejecutar el servidor, la API y los clientes desde cero.

Tareas:
Documentar la arquitectura cliente-servidor.
Explicar cómo configurar la IP y el puerto TCP.
Documentar cómo iniciar la API y MySQL.
Documentar la creación y configuración de la base de datos.
Explicar cómo ejecutar los tres clientes.
Documentar las dependencias y las versiones de .NET.
Agregar instrucciones para solucionar errores de conexión.
Incluir capturas de las tres computadoras conectadas y de una partida terminada.

MARTINEZ SANCHEZ MAYRA ALIN

implementar clase Jugador
Crear la clase que represente a cada uno de los tres jugadores.

Tareas:
Crear Jugador.cs.
Agregar ID.
Agregar nombre.
Agregar lista de cartas.
Agregar métodos para agregar cartas.
Agregar métodos para quitar cartas.
Probar los tres jugadores.

implementar clase JuegoUNO
Crear la clase principal encargada de controlar el estado de la partida.

Tareas:
Crear JuegoUNO.cs.
Agregar lista de jugadores.
Controlar al jugador actual.
Controlar la carta actual.
Controlar dirección del juego.
Crear método para iniciar partida.
Crear método para cambiar turnos.

implementar validación de cartas
Implementar las reglas que determinan si una carta puede ser jugada.

Tareas:
Permitir carta del mismo color.
Permitir carta del mismo número.
Permitir carta del mismo símbolo.
Permitir comodín.

Validar correctamente +4.
Mostrar/retornar si una carta es válida.

implementar cartas especiales

Implementar el comportamiento de las cartas especiales del UNO.

Tareas:
Implementar Reversa.
Implementar Salta.
Implementar +2.
Implementar Comodín.
Implementar +4.
Cambiar de dirección cuando corresponda.
Saltar turno cuando corresponda.
Hacer robar cartas cuando corresponda.

implementar sistema de turnos

Implementar el sistema de turnos para los tres jugadores.

Tareas:

Jugador 1 → Jugador 2 → Jugador 3.
Cambiar correctamente de jugador.
Permitir cambiar de dirección.
Controlar turnos después de cartas especiales.
Evitar que un jugador juegue fuera de su turno.

implementar acción de robar

Implementar la acción de robar cartas durante el turno.

Tareas:
Robar una carta cuando el jugador no pueda jugar.
Agregar la carta a su mano.
Comprobar si puede jugar la carta robada.
Registrar la acción para posteriormente enviarla al log.

implementar UNO y penalización

Implementar la lógica de la regla UNO, incluyendo la detección de jugadores con una sola carta, la declaración de UNO y la penalización de robar dos cartas cuando un jugador no declare UNO dentro de la oportunidad correspondiente.

Tareas:
Detectar cuando un jugador queda con una carta.
Indicar que debe declarar UNO.
Crear estado para saber si declaró UNO.
Validar la declaración de UNO.
Determinar cuándo termina la oportunidad de declarar UNO.
Detectar que un jugador no declaró UNO.
Aplicar penalización de 2 cartas.
Obtener las 2 cartas del mazo.
Agregar las cartas a la mano del jugador.

Registrar la penalización para el log.
Reiniciar el estado de UNO para el siguiente turno.

detectar ganador

Implementar la condición de victoria y finalización de partida.

Tareas:
Detectar jugadores sin cartas.
Finalizar la partida.
Identificar al ganador.
Identificar jugadores perdedores.
Preparar información para guardar resultados en BD

implementa cartas de interfaz

Diseñar y vincular la representación lógica de las cartas con sus elementos visuales correspondientes en el formulario de la interfaz del usuario

Tareas:
Asociar las propiedades de la clase Carta con imágenes e íconos correspondientes
Implementar métodos de renderizado para mostrar mano actual del jugador
Vincular eventos de clic en cartas con lógica de validación
Actualizar el estado visual de la mesa y la pila de descarte en cada jugada

cambios pendientes de realizar

Ajustes lógicos y optimizaciones de código requeridos para corregir inconsistencias durante las pruebas de integración del juego

Tareas:
Refactorizar métodos de cambio de turno para gestionar correctamente los efectos combinados de cartas (+2 y +4)
Corregir errores en la detección de fin de juego cuando se roba la última carta del mazo
Ajustar el tiempo de espera de los turnos de jugadores para sincronizar efectos visuales
Resolver conflictos de fusiones en ramas de Git

animaciones de cartas

Añadir retroalimentación visual al usuario durante las acciones principales de la partida

Tareas:
Crear clases y helpers para animar el desplazamiento de las cartas del mazo a la mano del jugador
Implementar animación de deslizamiento/transición al colocar una carta en la pila de descarte
Agregar efectos visuales al presionar el botón “UNO” o cuando se aplica una penalización

crear el proyecto independiente del servidor TCP

Crear un proyecto C# independiente llamado ProyectoUno.Servidor, encargado de aceptar las conexiones de las tres computadoras mediante sockets TCP

Tareas:
Crear el proyecto de servidor.
Configurar el puerto TCP 11000.

Utilizar Socket, SocketType.Stream y ProtocolType.Tcp.
Configurar el socket para escuchar en las interfaces de red disponibles.
Aceptar hasta tres conexiones de jugadores.
Mantener las conexiones abiertas durante la partida.
Detectar desconexiones y liberar los sockets correctamente.
Mostrar en consola el estado de cada conexión.

definir el protocolo de comunicación entre cliente y servidor

Establecer un formato común para todas las solicitudes y respuestas que viajen mediante sockets TCP.

Tareas:
Definir mensajes para conectar, jugar una carta, robar, declarar UNO y desconectar.
Definir respuestas de aceptación, rechazo y actualización de partida.
Utilizar JSON como formato de los mensajes.
Definir cómo se delimita cada mensaje, por ejemplo, una línea JSON terminada en salto de línea.
Implementar lectura de mensajes completos sin asumir que cada ReceiveAsync corresponde a un mensaje.
Manejar mensajes inválidos y errores de deserialización.
Acordar los nombres y tipos de las propiedades que usarán ambos proyectos.

implementar la asignación de los tres jugadores y gestionar la sala de partida

Implementar la lógica para que cada conexión corresponda a un jugador distinto y todos participen en la misma partida.

Tareas:
Asignar un identificador único a cada conexión.
Registrar el nombre de cada jugador.
Permitir únicamente tres participantes en la partida.
Esperar a que se conecten los tres antes de iniciar.
Crear una sola instancia de la partida compartida.
Rechazar conexiones adicionales mientras la sala esté llena.
Definir el comportamiento cuando un jugador se desconecta.

trasladar el motor del UNO al servidor

Adaptar las clases existentes para que el servidor controle el mazo, las manos, la carta central, los turnos y el estado de la partida.

Tareas:
Reutilizar las clases actuales siempre que sea posible.
Crear y barajar un único mazo para la partida.
Repartir las cartas iniciales a los tres jugadores.
Mantener las manos completas únicamente en el servidor.
Guardar la carta superior del descarte.
Mantener el jugador actual y el sentido del juego.
Garantizar que las acciones se procesen de una en una.
Evitar que cada cliente cree una partida local independiente.

implementar las reglas y validar las jugadas en el servidor

Trasladar la validación de las jugadas al servidor para que ningún cliente pueda ejecutar movimientos inválidos o saltarse los turnos.

Tareas:
Validar el turno del jugador.
Validar color, número y símbolo.
Validar comodines y la elección de color.
Implementar reversa y salto.
Aplicar las penalizaciones de +2 y +4 según las reglas del proyecto.
Implementar la acción de robar.
Implementar la declaración y penalización por UNO.
Detectar al ganador y finalizar la partida.
Rechazar acciones inválidas sin modificar el estado.

enviar estados privados y públicos de la partida
Implementar las actualizaciones que el servidor enviará a cada cliente después de iniciar la partida o procesar una acción.

Tareas:
Enviar a cada cliente únicamente su propia mano.
Enviar la carta superior del descarte.
Enviar el jugador que tiene el turno.
Enviar los nombres y las cantidades de cartas de los otros jugadores.
Informar si el juego terminó y quién ganó.
Generar un estado diferente para cada cliente.
Evitar incluir las cartas privadas de otros jugadores en el JSON.
Enviar las actualizaciones después de cada acción válida.

configurar la API para ejecutarse en la red
Adaptar ProyectoUno.API para que el servidor pueda utilizarla y para que sus endpoints funcionen con la configuración real de ejecución.

Tareas:
Revisar las URL de escucha de la API.
Separar la dirección de escucha de la dirección usada por los clientes.
Configurar la URL de la API que utilizará el servidor.
Mantener las credenciales de MySQL fuera del código fuente compartido.
Verificar el acceso a la base de datos desde la máquina de la API.
Configurar el firewall según la topología elegida.
Probar los endpoints existentes.

integrar las reglas especiales y sincronizar las tres interfaces
Verificar que las acciones especiales se procesen en el servidor y que todos los clientes reciban el resultado correcto.

Tareas:
Probar reversa con tres jugadores.
Probar salto de turno.
Probar +2 y comprobar quién recibe las cartas.
Probar comodín y selección de color.

Probar +4 según las reglas implementadas.
Probar la declaración de UNO y su penalización.
Probar robo, descarte y fin de partida.
Verificar que no se procesen dos jugadas simultáneas como si ambas fueran válidas.
Confirmar que las tres interfaces se actualicen después de cada acción.

implementar el juego completo en tres computadoras independientes
Realizar pruebas de integración con tres clientes Windows Forms conectados al mismo servidor TCP y con la API y MySQL activos.

Tareas:
Conectar tres computadoras a la misma red.
Iniciar el servidor TCP.
Iniciar la API y comprobar la conexión con MySQL.
Conectar los tres clientes.
Verificar el reparto inicial de cartas.
Confirmar que cada jugador solo ve su propia mano.
Comprobar los turnos y las reglas especiales.
Probar una desconexión inesperada.
Terminar una partida y revisar los registros en MySQL.
Corregir los errores detectados y guardar evidencia de las pruebas.

documentar la instalación, configuración y ejecución del UNO en red
Actualizar la documentación del proyecto para que el profesor pueda ejecutar el servidor, la API y los clientes desde cero.

Tareas:
Documentar la arquitectura cliente-servidor.
Explicar cómo configurar la IP y el puerto TCP.
Documentar cómo iniciar la API y MySQL.
Documentar la creación y configuración de la base de datos.
Explicar cómo ejecutar los tres clientes.
Documentar las dependencias y las versiones de .NET.
Agregar instrucciones para solucionar errores de conexión.
Incluir capturas de las tres computadoras conectadas y de una partida terminada.

RODRIGUEZ DE LA TORRE AARON EMMANUEL

implementar interfaz principal
Crear la interfaz gráfica del juego en Windows Forms.

Tareas:
Crear formulario principal.
Mostrar los tres jugadores.
Mostrar las cartas de cada jugador.
Mostrar carta actual.
Mostrar turno actual.

Crear botones necesarios.
Crear mensaje de ganador.

mostrar cartas como botones

Implementar la representación visual de las cartas de los jugadores.

Tareas:
Mostrar las cartas de Jugador 1.
Mostrar las cartas de Jugador 2.
Mostrar las cartas de Jugador 3.
Permitir seleccionar una carta.
Actualizar la interfaz después de jugar.
Actualizar la interfaz después de robar.
Agregar botón UNO.
Habilitar/deshabilitar el botón según el estado de la partida.
Conectar el botón con el método de declaración de UNO.
Actualizar la interfaz después de declarar UNO.
Actualizar la interfaz si se aplican las 2 cartas de penalización.

crear base de datos

Crear la base de datos necesaria para almacenar la información solicitada por el proyecto.

Tareas:
Crear tabla Jugador.
Crear tabla Partida.
Crear tabla ResultadoPartida.
Crear tabla LogJuego.
Definir llaves primarias.
Definir llaves foráneas.
Crear relaciones.
Insertar los tres jugadores iniciales.

crear conexión con BD

Implementar la conexión entre el programa C# y la base de datos.

Tareas:
Crear ConexionBD.cs.
Configurar conexión.
Probar conexión.
Manejar errores de conexión.
Crear métodos básicos para ejecutar consultas.

DAO de jugadores

Crear las operaciones necesarias para consultar la información de los jugadores.

Tareas:
Crear JugadorDAO.cs.
Consultar jugadores.
Obtener jugador por ID.
Consultar estadísticas del jugador.

guardar historial de partidas

Implementar el almacenamiento de los resultados de cada partida.

Tareas:
Guardar partida.
Guardar ganador.
Guardar jugadores participantes.
Guardar resultado de cada jugador.
Registrar ganadas/perdidas.
Consultar historial.

implementar Log del juego

Registrar en la base de datos todos los movimientos realizados durante una partida.

Tareas:
Crear LogJuegoDAO.cs.
Registrar cartas jugadas.
Registrar cartas robadas.
Registrar cambios de color.
Registrar turnos.
Registrar acciones de cartas especiales.
Guardar fecha/hora.
Asociar cada movimiento con jugador y partida.

conectar interfaz con BD

Integrar la interfaz gráfica con las funciones de base de datos.

Tareas:
Guardar partida al finalizar.
Guardar resultados.
Guardar movimientos.
Mostrar información del jugador cuando sea necesario.
Verificar que los datos realmente se guarden.

animaciones de cartas

Optimizar el refresh de pantalla para evitar el parpadeo mediante técnicas de Double Buffering

API

Implementar API al proyecto

Endpoints principales:
GET /api/jugadores: obtiene el catálogo de jugadores registrados y sus estadísticas generales
POST /api/partidas: registra el inicio de una nueva partida y almacena sus parámetros de configuración
POST /api/partidas/resultados: guarda los resultados finales, incluyendo el ganador y los puntajes de los perdedores
POST /api/log: registra de forma asíncrona cada evento o movimiento dentro del juego (cartas jugadas, robadas, penalizaciones y declaraciones de UNO)

La comunicación se realiza mediante objetos DTO (Data Transfer Objects) (PartidaDTO, LogJuegoDTO), garantizando la integridad de la información

definir el protocolo de comunicación entre cliente y servidor

Establecer un formato común para todas las solicitudes y respuestas que viajen mediante sockets TCP.

Tareas:
Definir mensajes para conectar, jugar una carta, robar, declarar UNO y desconectar.
Definir respuestas de aceptación, rechazo y actualización de partida.
Utilizar JSON como formato de los mensajes.
Definir cómo se delimita cada mensaje, por ejemplo, una línea JSON terminada en salto de línea.
Implementar lectura de mensajes completos sin asumir que cada ReceiveAsync corresponde a un mensaje.
Manejar mensajes inválidos y errores de deserialización.
Acordar los nombres y tipos de las propiedades que usarán ambos proyectos.

implementar ClienteSocket.cs en Windows Forms

Crear una clase responsable de conectar cada cliente con el servidor TCP y mantener la comunicación durante toda la partida.

Tareas:
Conectar con la IP del servidor y el puerto 11000.
Enviar el nombre del jugador al conectarse.
Recibir la confirmación de conexión y el identificador asignado.
Enviar y recibir mensajes usando el protocolo acordado.
Mantener una recepción continua de mensajes.
Detectar desconexiones y mostrar mensajes en la interfaz.
Implementar un método para desconectarse correctamente.
Evitar bloquear el hilo de la interfaz mientras se reciben datos.

enviar estados privados y públicos de la partida

Implementar las actualizaciones que el servidor enviará a cada cliente después de iniciar la partida o procesar una acción.

Tareas:
Enviar a cada cliente únicamente su propia mano.
Enviar la carta superior del descarte.
Enviar el jugador que tiene el turno.
Enviar los nombres y las cantidades de cartas de los otros jugadores.
Informar si el juego terminó y quién ganó.
Generar un estado diferente para cada cliente.
Evitar incluir las cartas privadas de otros jugadores en el JSON.
Enviar las actualizaciones después de cada acción válida.

modificar InicioForms.cs para conectarse al servidor

Adaptar la pantalla de inicio para que cada computadora introduzca únicamente el nombre de su jugador y se conecte a la partida compartida.

Tareas:

Conservar el diseño actual de la pantalla.
Permitir introducir el nombre del jugador.
Configurar o introducir la IP del servidor.
Conectar mediante ClienteSocket.cs.
Mostrar los estados de conexión y espera.
Abrir UNO.cs cuando el servidor confirme que los tres jugadores están listos.
Mostrar un mensaje si la sala está llena o falla la conexión.
Eliminar la creación local de los tres jugadores desde el formulario.

adaptar UNO.cs para usar el estado recibido por sockets

Modificar la interfaz principal para que deje de administrar las reglas localmente y utilice las actualizaciones que envía el servidor.

Tareas:
Conservar las cartas de Snoopy, los fondos y las animaciones.
Eliminar la inicialización local del mazo y de la partida.
Actualizar la mano del jugador al recibir ESTADO_PARTIDA.
Mostrar las cantidades de cartas de los otros dos jugadores.
Mostrar el turno actual y la carta superior del descarte.
Desactivar las acciones cuando no sea el turno del jugador.
Mostrar errores de jugadas rechazadas.
Actualizar la interfaz sin bloquear el hilo principal.
Mostrar el ganador y finalizar la partida cuando lo indique el servidor.

adaptar AbanicoCartas.cs para mostrar únicamente la mano propia

Conectar el control visual de las cartas con la mano privada recibida del servidor y conservar el diseño actual.

Tareas:
Mostrar las cartas que recibe el jugador local.
Conservar el comportamiento del abanico y los clics.
Enviar al servidor la selección de la carta.
Mostrar los reversos de las cartas ajenas.
Mostrar correctamente la cantidad de cartas de cada rival.
Evitar que los controles visuales accedan a las manos completas.
Conservar las imágenes, el tamaño y la orientación de las cartas.

enviar las acciones de juego desde los controles de Windows Forms

Conectar los botones de la interfaz con el protocolo de mensajes para que las acciones se procesen en el servidor.

Tareas:
Enviar JUGAR_CARTA al seleccionar una carta.
Enviar ROBAR_CARTA al pulsar Robar.
Enviar DECLARAR_UNO al pulsar UNO.
Enviar la elección de color después de un comodín.
Mostrar el resultado que envía el servidor.
Evitar que una acción se envíe repetidamente por varios clics.
No modificar directamente las listas de cartas del juego desde los botones.

integrar las reglas especiales y sincronizar las tres interfaces

Verificar que las acciones especiales se procesen en el servidor y que todos los clientes reciban el resultado correcto.

Tareas:
Probar reversa con tres jugadores.
Probar salto de turno.
Probar +2 y comprobar quién recibe las cartas.
Probar comodín y selección de color.
Probar +4 según las reglas implementadas.
Probar la declaración de UNO y su penalización.
Probar robo, descarte y fin de partida.
Verificar que no se procesen dos jugadas simultáneas como si ambas fueran válidas.
Confirmar que las tres interfaces se actualicen después de cada acción.

Integrar el servidor TCP con la API y MySQL

Conectar la lógica de la partida con el registro persistente, para que cada partida y su resultado queden almacenados correctamente.

Tareas:
Registrar la partida al comenzar.
Relacionar cada jugador con su participación.
Registrar los movimientos aceptados.
Registrar robos y efectos especiales.
Registrar el ganador al terminar.
Manejar errores HTTP sin duplicar operaciones.
Verificar los datos guardados en MySQL.
Comprobar que el servidor utiliza el identificador correcto de la partida.

implementar el juego completo en tres computadoras independientes

Realizar pruebas de integración con tres clientes Windows Forms conectados al mismo servidor TCP y con la API y MySQL activos.

Tareas:
Conectar tres computadoras a la misma red.
Iniciar el servidor TCP.
Iniciar la API y comprobar la conexión con MySQL.
Conectar los tres clientes.
Verificar el reparto inicial de cartas.
Confirmar que cada jugador solo ve su propia mano.
Comprobar los turnos y las reglas especiales.
Probar una desconexión inesperada.
Terminar una partida y revisar los registros en MySQL.
Corregir los errores detectados y guardar evidencia de las pruebas.

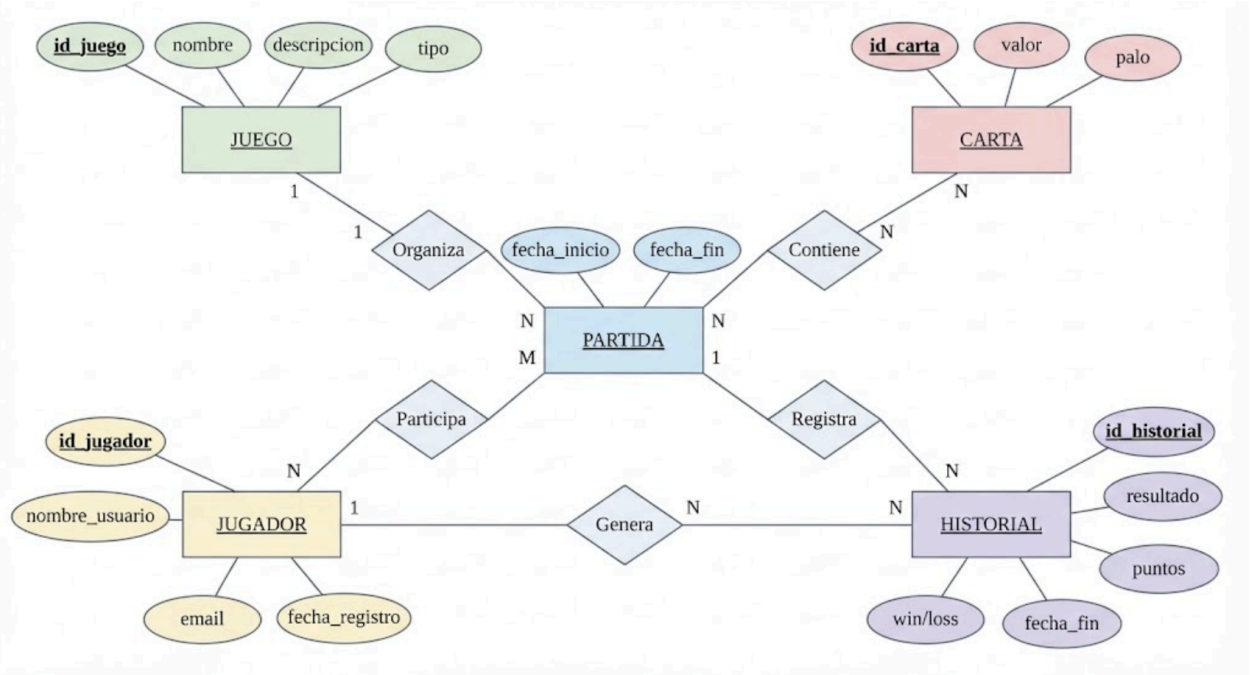
documentar la instalación, configuración y ejecución del UNO en red

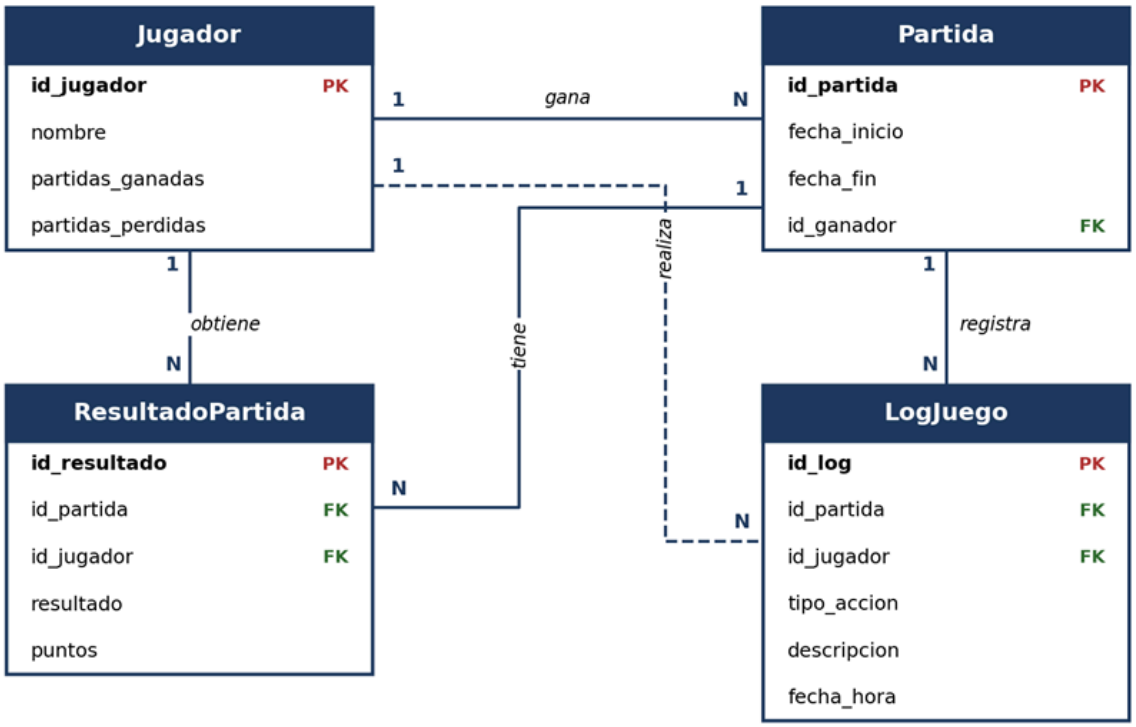
Actualizar la documentación del proyecto para que el profesor pueda ejecutar el servidor, la API y los clientes desde cero.

Tareas:

Documentar la arquitectura cliente-servidor.
Explicar cómo configurar la IP y el puerto TCP.
Documentar cómo iniciar la API y MySQL.
Documentar la creación y configuración de la base de datos.
Explicar cómo ejecutar los tres clientes.
Documentar las dependencias y las versiones de .NET.
Agregar instrucciones para solucionar errores de conexión.
Incluir capturas de las tres computadoras conectadas y de una partida terminada.

- Diagrama entidad-relación de la base de datos



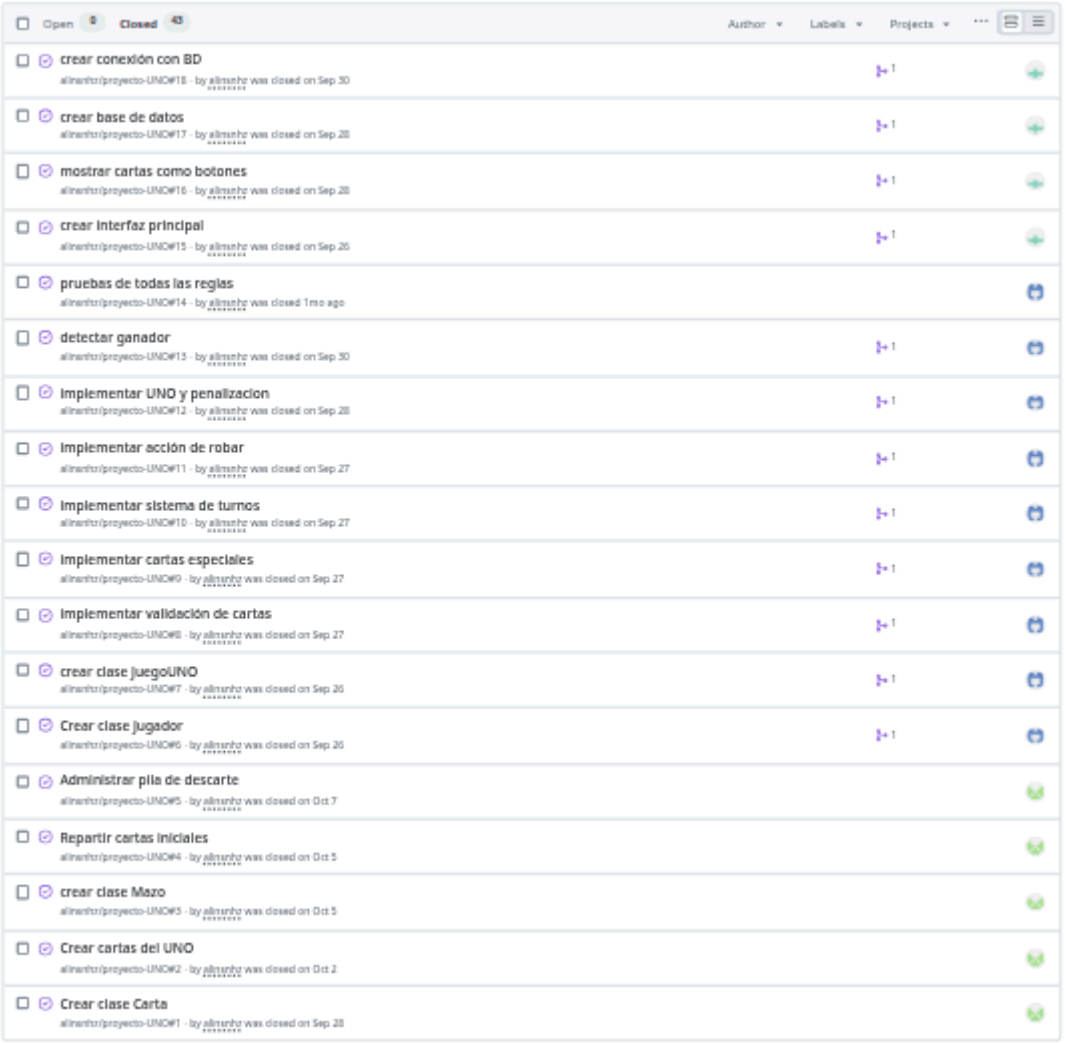


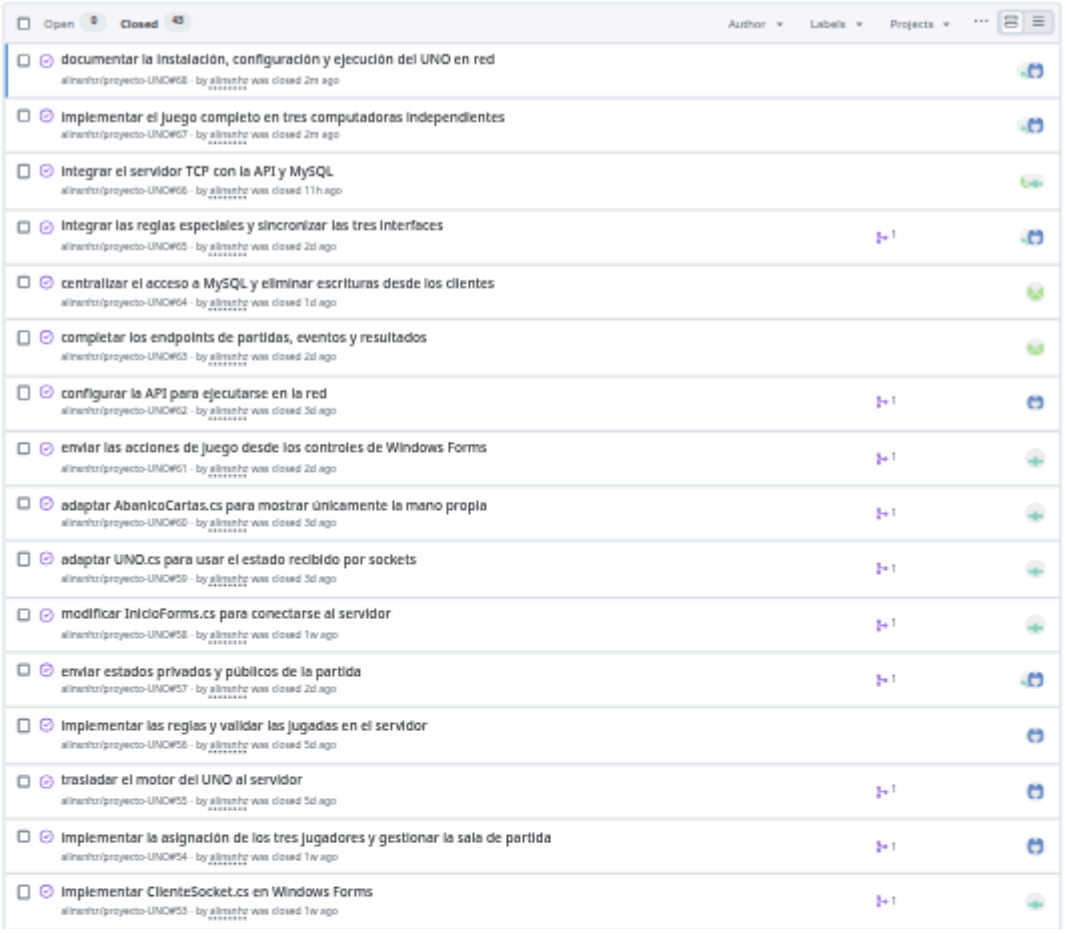
- Listado de commits

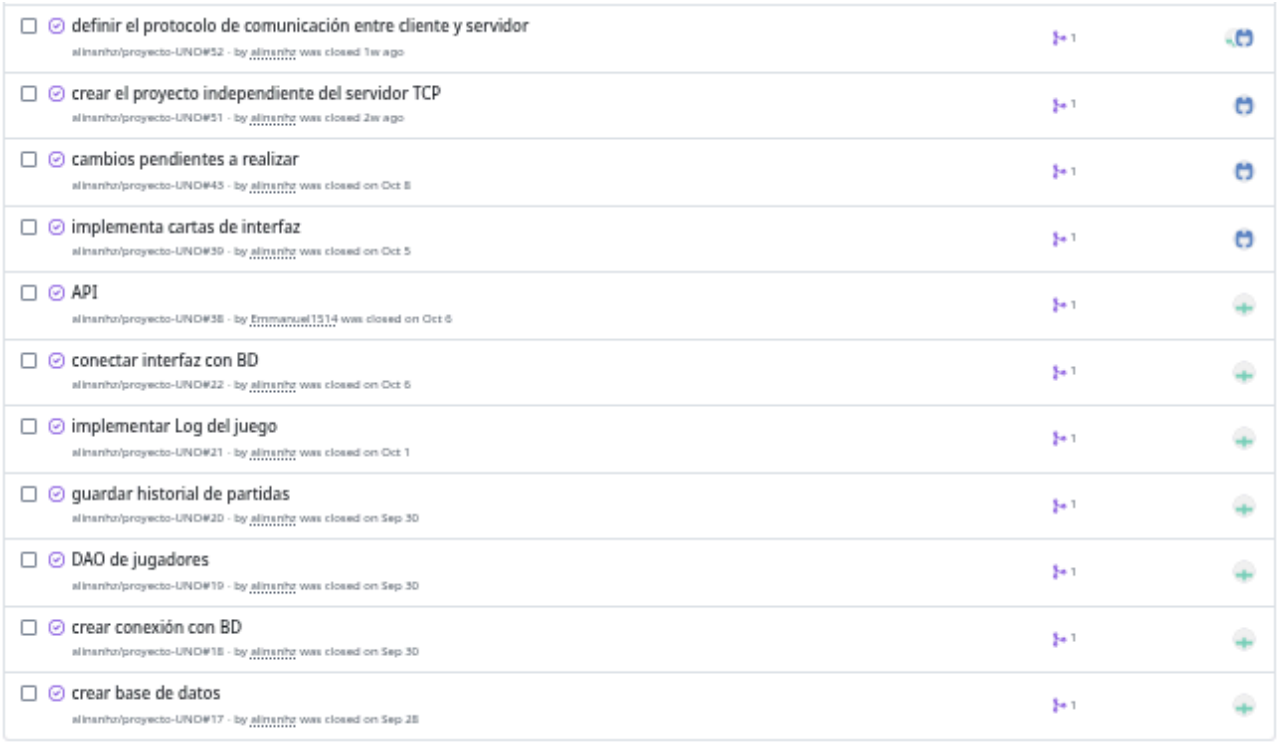
La división de las tareas del proyecto se organizó mediante issues en GitHub, en las que se describen las actividades asignadas a cada integrante del grupo. Cada issue correspondía a una tarea específica y daba origen a una branch propia, lo que permitía desarrollar cada funcionalidad de forma independiente. Esta organización se corresponde con la división de tareas presentada en el apartado 4, facilitando la identificación de las responsabilidades de cada integrante y el seguimiento del progreso del proyecto.

En total, se crearon 43 issues, que guiaron el desarrollo de las funcionalidades previstas. El flujo de trabajo consistió en implementar cada tarea en su respectiva branch, registrar los cambios mediante commits y, una vez finalizada la tarea, integrar las modificaciones en la rama principal (main) mediante un merge.

Además de los commits relacionados directamente con la finalización de las issues, se realizaron otros destinados a corregir problemas identificados durante el desarrollo, ajustar la lógica y solucionar cuestiones relacionadas con el funcionamiento del sistema. Un ejemplo es el commit “43 cambios pendientes a realizar”, dedicado a los ajustes finales del trabajo, con el objetivo de perfeccionar los detalles de la implementación y garantizar el correcto funcionamiento del proyecto.




Open 0
Closed 43
Author
Labels
Projects
documentar la instalación, configuración y ejecución del UNO en red
alinanhz/proyecto-UNO#68 · by alinanhz was closed 2m ago
implementar el juego completo en tres computadoras independientes
alinanhz/proyecto-UNO#67 · by alinanhz was closed 2m ago
integrar el servidor TCP con la API y MySQL
alinanhz/proyecto-UNO#66 · by alinanhz was closed 11h ago
integrar las reglas especiales y sincronizar las tres interfaces
alinanhz/proyecto-UNO#65 · by alinanhz was closed 2d ago
centralizar el acceso a MySQL y eliminar escrituras desde los clientes
alinanhz/proyecto-UNO#64 · by alinanhz was closed 1d ago
completar los endpoints de partidas, eventos y resultados
alinanhz/proyecto-UNO#63 · by alinanhz was closed 2d ago
configurar la API para ejecutarse en la red
alinanhz/proyecto-UNO#62 · by alinanhz was closed 3d ago
enviar las acciones de juego desde los controles de Windows Forms
alinanhz/proyecto-UNO#61 · by alinanhz was closed 2d ago
adaptar AbanicoCartas.cs para mostrar únicamente la mano propia
alinanhz/proyecto-UNO#60 · by alinanhz was closed 3d ago
adaptar UNO.cs para usar el estado recibido por sockets
alinanhz/proyecto-UNO#59 · by alinanhz was closed 3d ago
modificar InicioForms.cs para conectarse al servidor
alinanhz/proyecto-UNO#58 · by alinanhz was closed 1w ago
enviar estados privados y públicos de la partida
alinanhz/proyecto-UNO#57 · by alinanhz was closed 2d ago
implementar las reglas y validar las jugadas en el servidor
alinanhz/proyecto-UNO#56 · by alinanhz was closed 5d ago
trasladar el motor del UNO al servidor
alinanhz/proyecto-UNO#55 · by alinanhz was closed 5d ago
implementar la asignación de los tres jugadores y gestionar la sala de partida
alinanhz/proyecto-UNO#54 · by alinanhz was closed 1w ago
implementar ClienteSocket.cs en Windows Forms
alinanhz/proyecto-UNO#53 · by alinanhz was closed 1w ago


definir el protocolo de comunicación entre cliente y servidor
alinanhz/proyecto-UNO#52 · by alinanhz was closed 1w ago
crear el proyecto independiente del servidor TCP
alinanhz/proyecto-UNO#51 · by alinanhz was closed 2w ago
cambios pendientes a realizar
alinanhz/proyecto-UNO#43 · by alinanhz was closed on Oct 8
implementa cartas de interfaz
alinanhz/proyecto-UNO#39 · by alinanhz was closed on Oct 5
API
alinanhz/proyecto-UNO#38 · by Emmanuel1514 was closed on Oct 6
conectar interfaz con BD
alinanhz/proyecto-UNO#22 · by alinanhz was closed on Oct 6
implementar Log del juego
alinanhz/proyecto-UNO#21 · by alinanhz was closed on Oct 1
guardar historial de partidas
alinanhz/proyecto-UNO#20 · by alinanhz was closed on Sep 30
DAO de jugadores
alinanhz/proyecto-UNO#19 · by alinanhz was closed on Sep 30
crear conexión con BD
alinanhz/proyecto-UNO#18 · by alinanhz was closed on Sep 30
crear base de datos
alinanhz/proyecto-UNO#17 · by alinanhz was closed on Sep 28

24/09/26 - 25/09/26

Commits on Sep 25, 2026

agrega nombre y lista de cartas de un jugador alinsnhz committed last week		8b66d54
agrega using de lista y declara variable ID a Jugador alinsnhz committed last week		68a86e8
crear clase Jugador alinsnhz committed last week		320bc2f
crear estructura proyecto UNO alinsnhz committed last week		de54feb

Commits on Sep 24, 2026

Initial commit Emmanuel1514 authored last week	Verified	c8c583c

Commits on Sep 25, 2026

agrega método para contabilizar cartas de un jugador alinsnhz committed last week	79e1c29
agrega método para quitar/eliminar cartas a un jugador alinsnhz committed last week	5be7b87
agrega método para añadir cartas a un jugador alinsnhz committed last week	cb89cb1
agrega constructor de Jugador alinsnhz committed last week	921fa29

26/09/26

Commits on Sep 26, 2026

crear método de validación de cartas alinsnhz committed 5 days ago		87f61b8
Mostrar cartas Emmanuel1514 committed 5 days ago		b954e20
Actualiza interfaz Emmanuel1514 committed 5 days ago		a141277
Panel del las cartas del jugador 3 Emmanuel1514 committed 5 days ago		56f15e0
cartas del jugador 2 Emmanuel1514 committed 5 days ago		3fde5f4
Se ponen las cartas como botones Aaron authored and Aaron committed 5 days ago		70bd8ee
15-crear-interfaz-principal Emmanuel1514 authored 5 days ago	Verified	7112f65
Merge branch '15-crear-interfaz-principal' of https://github.com/alinsnhz/proyecto-UNO into 15-crear-interfaz-principal Aaron authored and Aaron committed 5 days ago		f727e82
Se modifica fondo y se hace interfaz para mostrar las cartas y se muestra al player y su nombre Emmanuel1514 authored and Aaron committed 5 days ago		94df72e
Merge pull request #24 from alinsnhz/7-crear-clase-juegouno Emmanuel1514 authored 5 days ago	Verified	8d27b8e
agregar cambio de turno alinsnhz committed 5 days ago		e7ddc81
agrega método para iniciar partida alinsnhz committed 5 days ago		ab68605
agrega constructor para inicializar el estado del juego alinsnhz committed 5 days ago		3b53923
agrega control de la direccion de la jugada alinsnhz committed 5 days ago		6fe29b6

agregar control de carta actual
alinsnhz committed 5 days ago — 6e8a61e

agrega control de jugador actual
alinsnhz committed 5 days ago — 47865da

agrega clase jugadorUNO
alinsnhz committed 5 days ago — 2631385

Merge branch '7-crear-clase-juegouno' of https://github.com/alinsnhz/proyecto-UNO into 7-crear-clase-juegouno
alinsnhz committed 5 days ago — 3f0c981

agregar clase juegoUNO y lista de jugadores
alinsnhz committed 5 days ago — 654ea52

Merge pull request #23 from alinsnhz/6-crear-clase-jugador
Emmanuel1514 authored 5 days ago — Verified — 114ab64

27/09/26

agrega método para obtener jugador actual
alinsnhz committed 4 days ago — 8c5561c

resolución de conflictos
alinsnhz committed 4 days ago — 1e5d8aa

actualiza rama
alinsnhz committed 4 days ago — 4e73a25

Commits on Sep 27, 2026

Merge pull request #27 from alinsnhz/9-implementar-cartas-especiales
alinsnhz authored 4 days ago — Verified — 35c43fe

Merge branch '9-implementar-cartas-especiales' of https://github.com/alinsnhz/proyecto-UNO into 9-implementar-cartas-especiales
alinsnhz committed 4 days ago — 0f7a30a

agrega cambios de rama
alinsnhz committed 4 days ago — aa9c18a

Merge branch 'main' into 9-implementar-cartas-especiales
alinsnhz authored 4 days ago — Verified — e593998

Merge pull request #26 from alinsnhz/8-implementar-validación-de-cartas
alinsnhz authored 4 days ago — Verified — d1f0800

agrega método para regla +4
alinsnhz committed 4 days ago — 4c185b3

agrega método para implementar comodín de color
alinsnhz committed 4 days ago — 7f0f8a5

agrega método para la regla +2
alinsnhz committed 4 days ago — 24c673e

agrega método para la regla salto de turno
alinsnhz committed 4 days ago — b66f3e8

agrega método para aplicar reversa
alinsnhz committed 4 days ago — 4b5792a

pendiente de modificar
alinsnhz committed 4 days ago — a914611

28/09/26

Commits on Sep 28, 2026

Merge branch '1-crear-clase-carta'
brendabragaaaa committed 3 days ago — f101319

16-mostrar-cartas-como-botones
Emmanuel1514 authored 3 days ago — Verified — 9915829

Se hacen Botones de uno y Robar para los otros jugadores y ademas se añade Mensaje de UNO al momento que un jugador se queda con 1 carta
Emmanuel1514 committed 3 days ago — 2ca3e98

Se ajustan los tamaños de las cartas de los jugadores 2 y 3
Emmanuel1514 committed 3 days ago — 8666e23

Se ajustan los Botones de las cartas y de los jugadores
Emmanuel1514 committed 3 days ago — 7c6405b

Commits on Sep 28, 2026

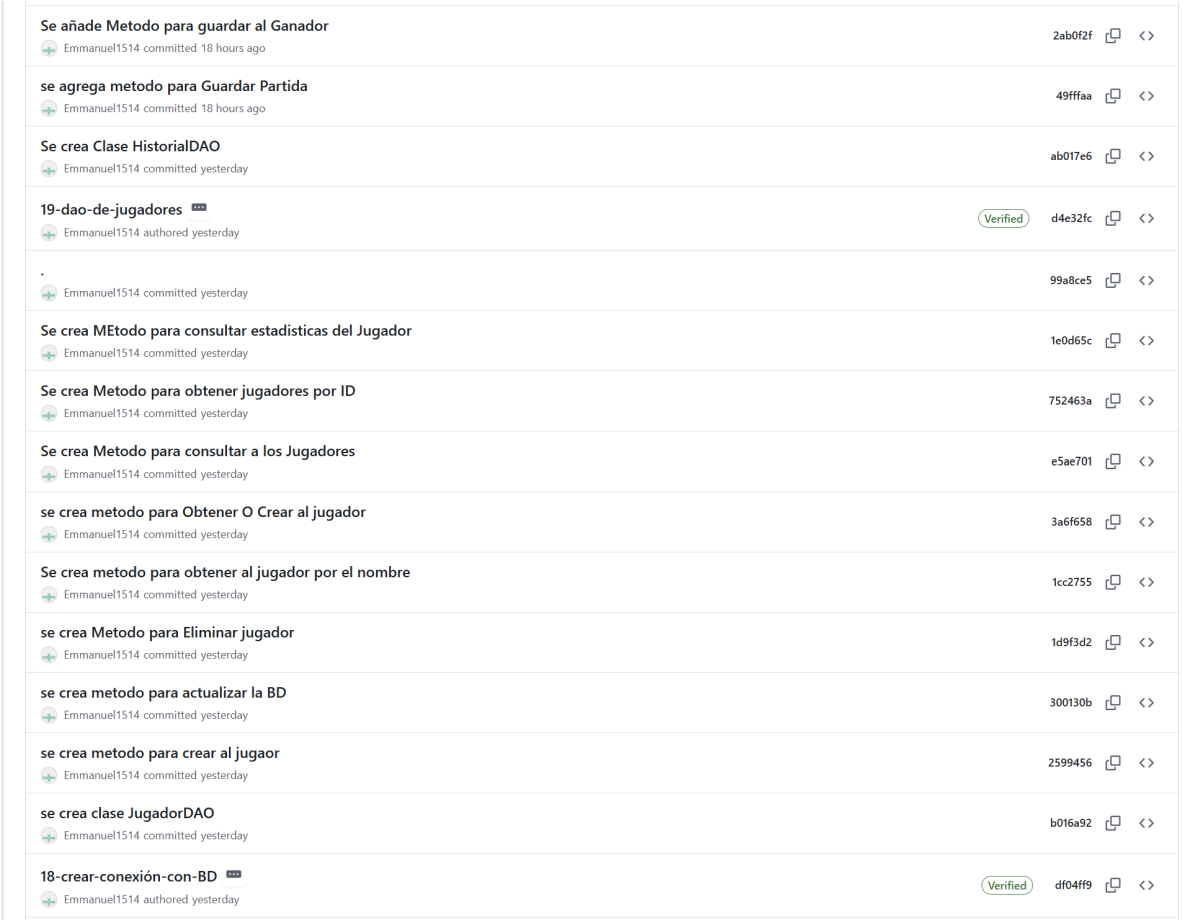
Crear clase Carta brendabragaaaa committed 3 days ago		e75503f
Merge pull request #32 from alinsnhz/12-implementar-uno alinsnhz authored 3 days ago	Verified	95c345f
Merge remote-tracking branch 'origin/main' into 12-implementar-uno alinsnhz committed 3 days ago		fc0f8d3
agrega método para procesar regla completa de UNO alinsnhz committed 3 days ago		3447394
agrega método para reiniciar estado de UNO alinsnhz committed 3 days ago		e3e6157
17-crear-base-de-datos Emmanuel1514 authored 3 days ago	Verified	a4ef2e9
Se crea la Base de Datos Emmanuel1514 committed 3 days ago		ca7ad30
agrega penalización de dos cartas por no declarar UNO alinsnhz committed 3 days ago		108b0c5
agrega método para penalizar falta de declaración de UNO alinsnhz committed 3 days ago		2cd0695
agrega control de oportunidad de declarar UNO alinsnhz committed 3 days ago		41865fb
agrega implementación de declaración de UNO alinsnhz committed 3 days ago		e0e1ca4
agrega método para indicar cuando el jugador debe declarar UNO alinsnhz committed 3 days ago		d5d0bc0
agrega método para detectar jugador con una carta alinsnhz committed 3 days ago		0496c4a
agrega estado de declaración de UNO alinsnhz committed 3 days ago		c6a56fb

30/09/26

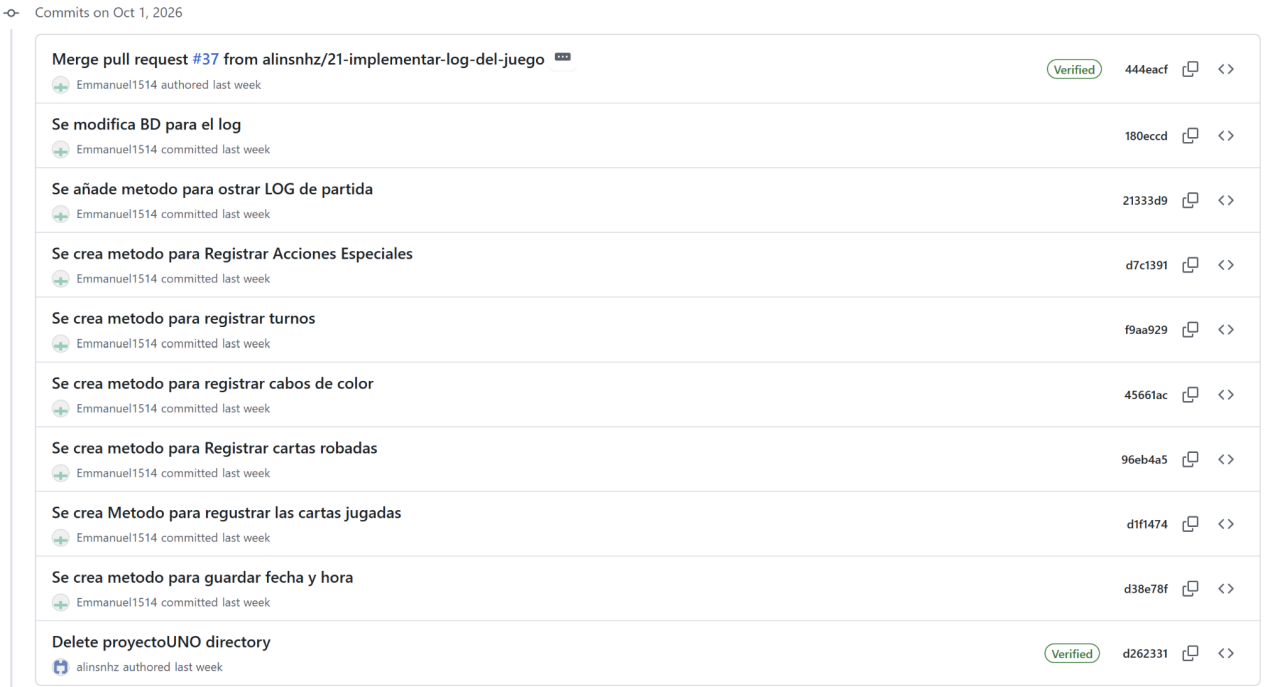
Commits on Sep 30, 2026

Merge pull request #36 from alinsnhz/13-detectar-ganador alinsnhz authored 15 hours ago	Verified	12d0433
agrega método para resultado en guardar para BD alinsnhz committed 15 hours ago		3a2a6cb
agrega lista de jugadores perdedores alinsnhz committed 15 hours ago		e184621
modifica para impedir acciones después de finalizar partida alinsnhz committed 15 hours ago		38e5396
agrega método para finalizar la partida cuando hay un ganador alinsnhz committed 15 hours ago		916eae0
agrega para guardar jugador ganador alinsnhz committed 15 hours ago		e9661ea
agrega método para detectar jugador sin cartas alinsnhz committed 15 hours ago		0a7dcdb
agrega estado de partida terminada alinsnhz committed 15 hours ago		10acecd
20-guardar-historial-de-partidas Emmanuel1514 authored 18 hours ago	Verified	a22fd8d
Se crea Metodo para Consultar Historial en la BD Emmanuel1514 committed 18 hours ago		ebdf469
Se agrega metodo para Registrar las partidad ganadas y perdidas Emmanuel1514 committed 18 hours ago		3cc9744
Se añade metodo para Guardar el Resultado de l aPartida Emmanuel1514 committed 18 hours ago		43a63c4
Se Añade metodo para Guardar los participantes Emmanuel1514 committed 18 hours ago		222de3e

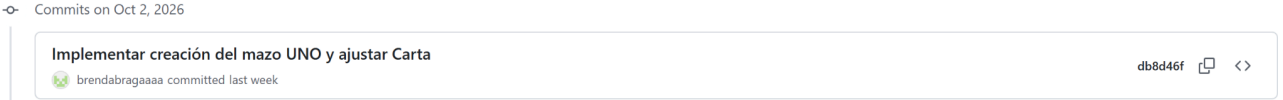
se crean métodos básicos para ejecutar consultas Emmanuel1514 committed yesterday		27a102a
Conexion con la BD Emmanuel1514 committed yesterday		6f07e52
Se hace coneccion con la BD y se instalan paquetes de MYSql.Data Emmanuel1514 committed yesterday		62a9a82
Se crea clase Conexion con BD Emmanuel1514 committed yesterday		116a86f
Se cambia de ubicacion la BD a la carpeta winfosforms1/windowsform1 Emmanuel1514 committed yesterday		ddbc25c



01/10/26



02/10/26



05/10/26

Commits on Oct 5, 2026

Merge pull request #40 from alinsnhz/39-implementa-cartas-de-interfaz alinsnhz authored 2 days ago	Verified	59fd0b3
Eliminar .vs/slnx.sqlite para resolver conflictos alinsnhz committed 2 days ago		6ba6144
modifica UNO.cs para implementar ruta de imágenes alinsnhz committed 2 days ago		5acdfd5
agrega cartas comodines alinsnhz committed 2 days ago		cf8fb01
agrega cartas azul alinsnhz committed 2 days ago		c78ec67
agrega cartas verde alinsnhz committed 2 days ago		0c8cbb6
agrega cartas amarillas y comodines de reversa y cancelar turno alinsnhz committed 3 days ago		1208a2f
agrega imagenes cartas rojas alinsnhz committed 3 days ago		86ad100
agrega carpeta para imagenes alinsnhz committed 3 days ago		d557d29
Implementar reparto inicial de cartas brendabragaaaa committed 3 days ago		7d39366
Crear clase Mazo para administrar el mazo brendabragaaaa committed 3 days ago		4322134

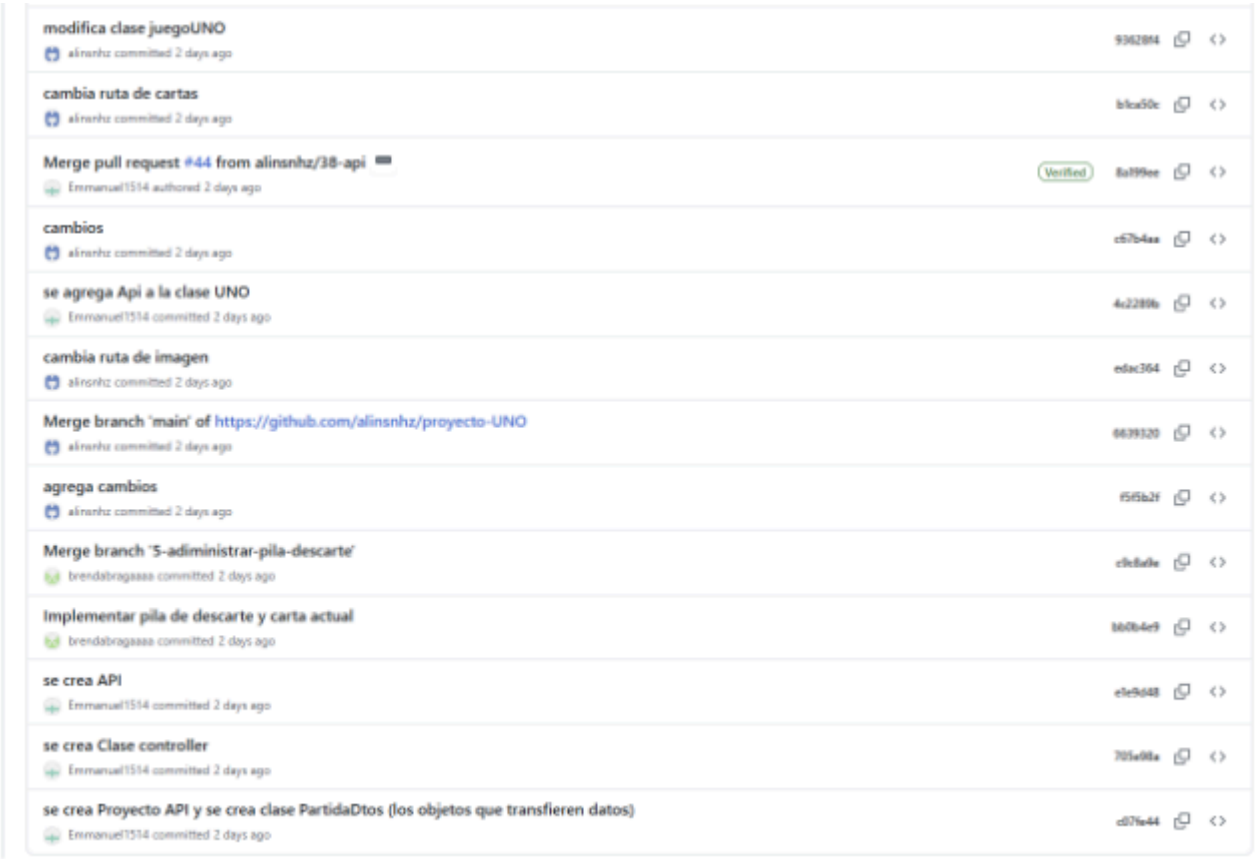
06/10/26

Commits on Oct 6, 2026

Merge pull request #42 from alinsnhz/22-conectar-interfaz-con-bd Emmanuel1514 authored 2 days ago	Verified	44ac931
Merge branch 'main' into 22-conectar-interfaz-con-bd Emmanuel1514 authored 2 days ago	Verified	274baa1
Se conecta la interfaz con la bd Emmanuel1514 committed 2 days ago		df67918
se modifica LOGjuegoDAO Emmanuel1514 committed 2 days ago		f436efd
cambios ?? Emmanuel1514 committed 2 days ago		97c9c2b

Commits on Oct 6, 2026

modifica sql alinsnhz committed 2 days ago		ba1664c
modifica clase UNO (otra vez) alinsnhz committed 2 days ago		df443dc
modifica clase UNO alinsnhz committed 2 days ago		e4c3589
cambios de clases alinsnhz committed 2 days ago		80aab2d
staged alinsnhz committed 2 days ago		242f177
resuelve conflictos alinsnhz committed 2 days ago		d041e6d
Merge pull request #45 from alinsnhz/43-cambios-pendientes-a-realizar alinsnhz authored 2 days ago	Verified	c77e6d4
Merge branch 'main' into 43-cambios-pendientes-a-realizar alinsnhz authored 2 days ago	Verified	da1edc0
modifica clase UNO y LogJuegoDAO alinsnhz committed 2 days ago		911c291



07/10/26

Commits on Oct 7, 2026

- agrega clases para animaciones de cartas — alinsnhz committed 18 hours ago — cbb48ed
- agrega forms de inicio — alinsnhz committed 19 hours ago — b049038
- agrega clase para abanico de Cartas — alinsnhz committed 19 hours ago — f8338bb
- agrega cambios de clases — alinsnhz committed 19 hours ago — 222eadd
- agrega clase Utilidades — alinsnhz committed 19 hours ago — 7f01aea
- Merge pull request #47 from alinsnhz/43-cambios-pendientes-a-realizar — alinsnhz authored yesterday — Verified — 445b35f
- agrega imagenes para iconos — alinsnhz committed yesterday — 80280c7
- Merge pull request #46 from alinsnhz/43-cambios-pendientes-a-realizar — alinsnhz authored yesterday — Verified — 03f8fd9
- cambios — alinsnhz committed yesterday — 291b348
- agrega cambios a clase Jugador y conexion — alinsnhz committed yesterday — c4cf493

08/10/26

Commits on Oct 8, 2026

modifica conflictos en clase UNO alinsnhz committed 2 days ago		0bcde0e
agrega cambios alinsnhz committed 2 days ago		9f68006
agrega iconos y cambio de clases alinsnhz committed 2 days ago		640dc12
Merge pull request #49 from alinsnhz/43-cambios-pendientes-a-realizar alinsnhz authored 2 days ago	Verified	baf637b
resuelve conflictos alinsnhz committed 2 days ago		acef0bf
agrega random y verifica reglas del juego alinsnhz committed 2 days ago		c51087a
agrega cambio de iconos random alinsnhz committed 2 days ago		707c7a0
Merge pull request #48 from alinsnhz/43-cambios-pendientes-a-realizar alinsnhz authored 2 days ago	Verified	829421f
Merge branch 'main' into 43-cambios-pendientes-a-realizar alinsnhz authored 2 days ago	Verified	0bb738a
agrega ventana de inicio alinsnhz committed 2 days ago		a677066

09/10/26

Commits on Oct 9, 2026

Merge pull request #50 from alinsnhz/43-cambios-pendientes-a-realizar Emmanuel1514 authored yesterday	Verified	0e1e09b
Merge branch 'main' into 43-cambios-pendientes-a-realizar Emmanuel1514 authored yesterday	Verified	8fdf0db
. Emmanuel1514 committed yesterday		268813c
Se Documenta InicioForms Emmanuel1514 committed yesterday		40ca1d6
Se hace que al mostrar el mensaje del ganador se salga a la ventana principal y se añade icono Emmanuel1514 committed yesterday		24bc378
Se añade Doble bufer para evitar parpadeo Emmanuel1514 committed yesterday		f12c7d3

10/10/26

Commits on Oct 10, 2026

agrega documentación de clases alinsnhz committed 5 minutes ago		177fbde

26/10/2026

Commits on Oct 26, 2026

agrega mensajes.cs para solicitudes de clientes documentada alinsnhz committed 2 weeks ago		140016d
Merge pull request #69 from alinsnhz/51-crear-el-proyecto-independiente-del-servidor-tcp alinsnhz authored 2 weeks ago	Verified	bf790c6
modifica clase Program.cs alinsnhz committed 2 weeks ago		38ff06d
añade código socket para servidor alinsnhz committed 2 weeks ago		292bcc8
Merge branch 'main' of https://github.com/alinsnhz/proyecto-UNO alinsnhz committed 2 weeks ago		677c73c

31/10/2026

Commits on Oct 31, 2026

Merge pull request #70 from alinsnhz/53-implementar-clientesocketcs-en-windows-forms Emmanuel1514 authored last week	Verified	db41cd4
. Emmanuel1514 committed last week		3917dfb
se ajustan comentarios Emmanuel1514 committed last week		954fa1f
Se cambia clase ClienteSocket y se agregan metodos en InicioForms Emmanuel1514 committed last week		b84b0be
se crean Metodos de la clase Cliente Socket Emmanuel1514 committed last week		7f80fe1
Se crea Clase ClienteSocket Emmanuel1514 committed last week		e7646a2

01/11/2026

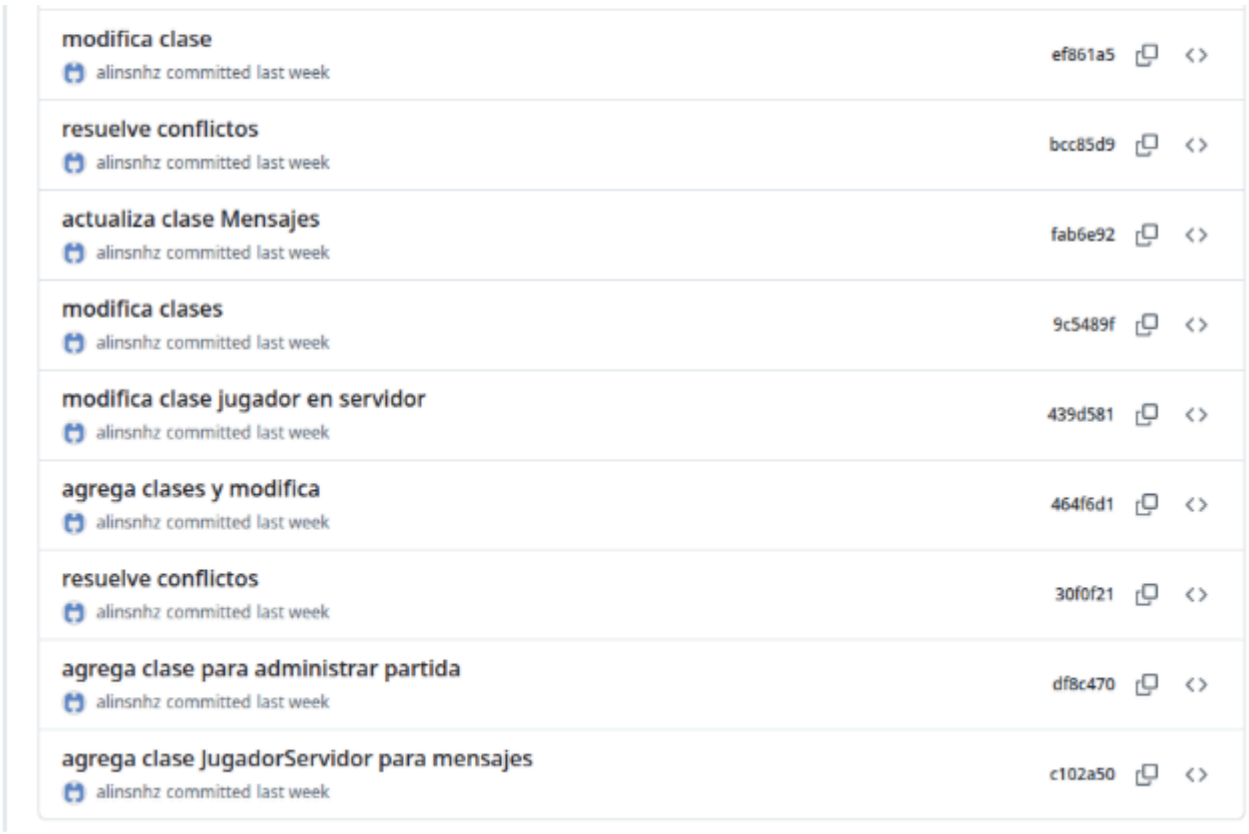
Commits on Nov 1, 2026

Merge pull request #72 from alinsnhz/58-modificar-inicioformscs-para-conectarse-al-servidor Emmanuel1514 authored last week	Verified	5442028
Se modifica InicioForms Emmanuel1514 committed last week		e6549e1
Merge pull request #71 from alinsnhz/52-definir-el-protocolo-de-comunicación-entre-cliente-y-servidor alinsnhz authored last week	Verified	7eaba8b
. alinsnhz committed last week		00674a0
agrega clases servidorSocket y estadoPartidaDto alinsnhz committed last week		027cb53

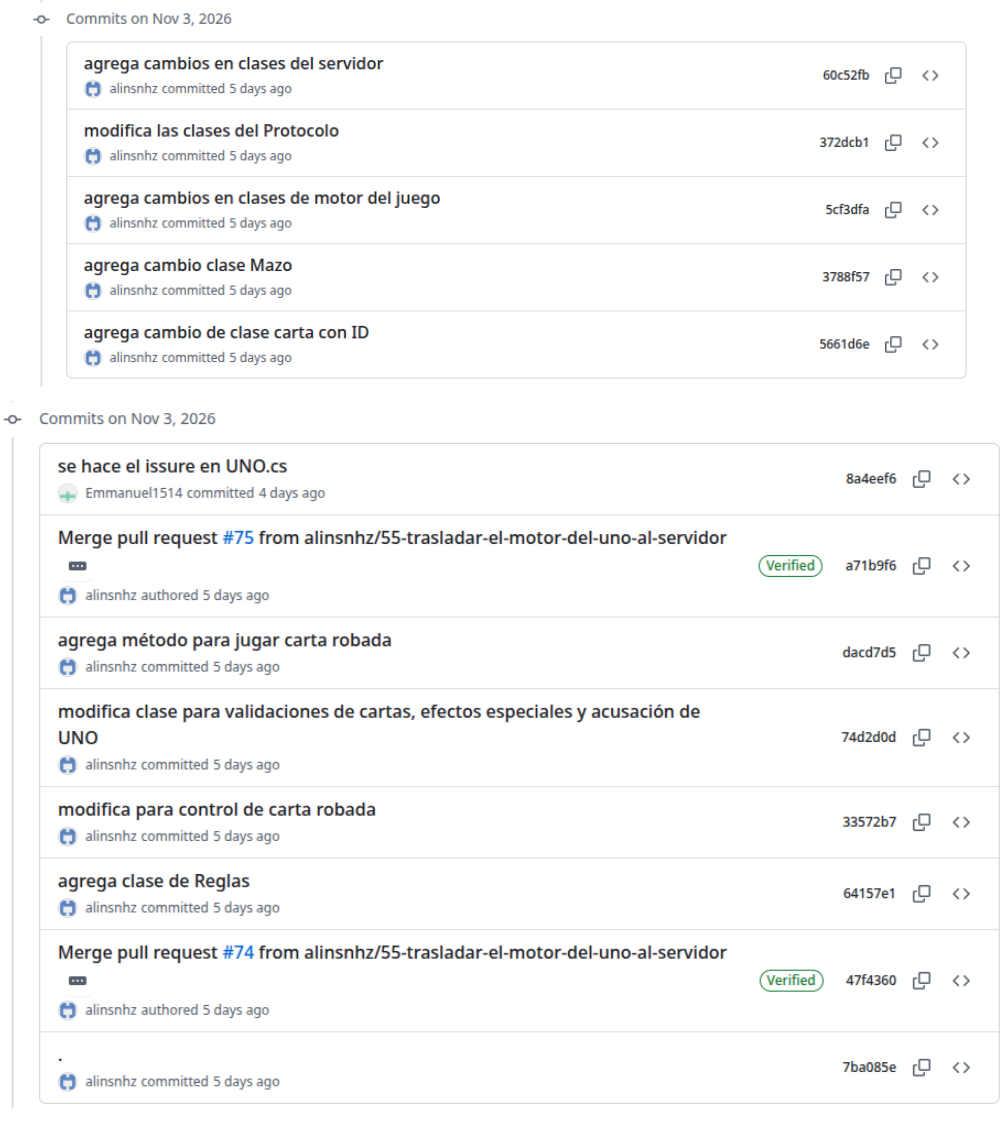
02/11/2026

Commits on Nov 2, 2026

resuelve conflictos alinsnhz committed last week		f07f11e
Merge pull request #73 from alinsnhz/54-implementar-la-asiganción-de-los-tres-jugadores-y-gestionar-la-sala-de-partida alinsnhz authored last week	Verified	cbce9e2
Merge branch 'main' into 54-implementar-la-asiganción-de-los-tres-jugadores-y-gestionar-la-sala-de-partida alinsnhz authored last week	Verified	a3d9463


modifica clase
alinsnhz committed last week
ef861a5
resuelve conflictos
alinsnhz committed last week
bcc85d9
actualiza clase Mensajes
alinsnhz committed last week
fab6e92
modifica clases
alinsnhz committed last week
9c5489f
modifica clase jugador en servidor
alinsnhz committed last week
439d581
agrega clases y modifica
alinsnhz committed last week
464f6d1
resuelve conflictos
alinsnhz committed last week
30f0f21
agrega clase para administrar partida
alinsnhz committed last week
df8c470
agrega clase JugadorServidor para mensajes
alinsnhz committed last week
c102a50

03/11/2026


Commits on Nov 3, 2026
agrega cambios en clases del servidor
alinsnhz committed 5 days ago
60c52fb
modifica las clases del Protocolo
alinsnhz committed 5 days ago
372dcb1
agrega cambios en clases de motor del juego
alinsnhz committed 5 days ago
5cf3dfa
agrega cambio clase Mazo
alinsnhz committed 5 days ago
3788f57
agrega cambio de clase carta con ID
alinsnhz committed 5 days ago
5661d6e
Commits on Nov 3, 2026
se hace el issure en UNO.cs
Emmanuel1514 committed 4 days ago
8a4eef6
Merge pull request #75 from alinsnhz/55-trasladar-el-motor-del-uno-al-servidor
Verified
alinsnhz authored 5 days ago
a71b9f6
agrega método para jugar carta robada
alinsnhz committed 5 days ago
dacd7d5
modifica clase para validaciones de cartas, efectos especiales y acusación de UNO
alinsnhz committed 5 days ago
74d2d0d
modifica para control de carta robada
alinsnhz committed 5 days ago
33572b7
agrega clase de Reglas
alinsnhz committed 5 days ago
64157e1
Merge pull request #74 from alinsnhz/55-trasladar-el-motor-del-uno-al-servidor
Verified
alinsnhz authored 5 days ago
47f4360
.
alinsnhz committed 5 days ago
7ba085e

04/11/2026

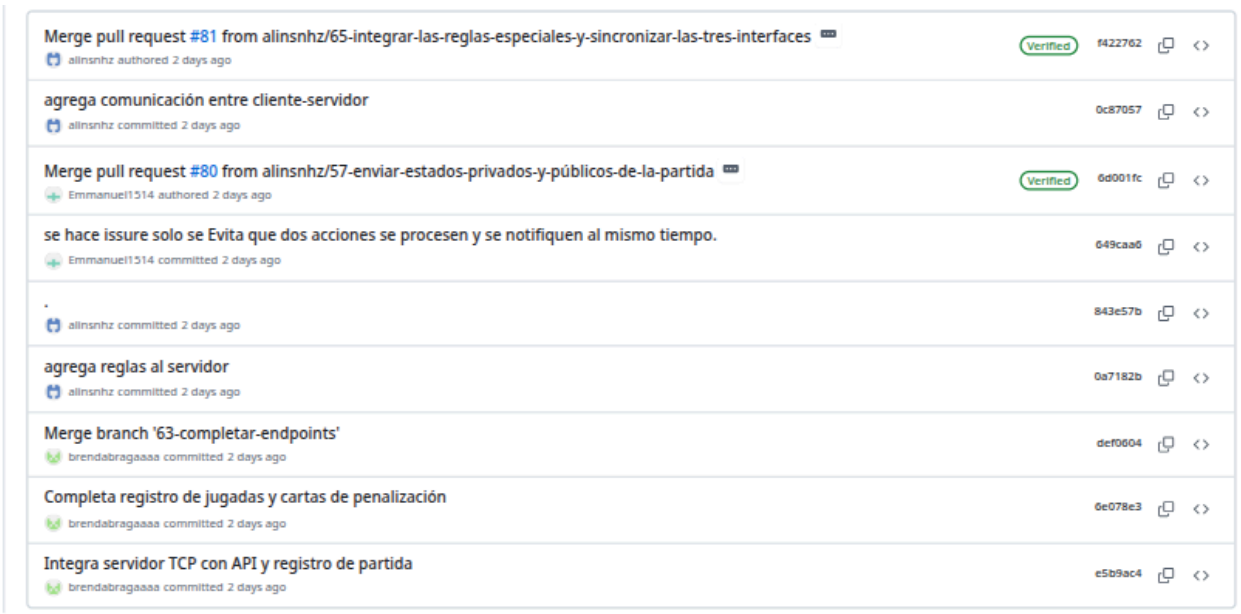
Commits on Nov 4, 2026

soluciona errores de constructor alinanhz committed 3 days ago	e0b1156	
Merge pull request #78 from alinsnhz/62-configurar-la-api-para-ejecutarse-en-la-red alinanhz authored 3 days ago	Verified	75b0416
resuelve conflictos alinanhz committed 3 days ago	97fa5ac	
resuelve conflictos alinanhz committed 3 days ago	a5b0903	
agrega configuración de API con Servidor alinanhz committed 3 days ago	ca865d7	
60-adaptar-abanicocartascs-para-mostrar-únicamente-la-mano-propia Emmanuel1514 authored 3 days ago	Verified	768dc0c
Se adapta que se muestre el reverso de las cartas a los otros players Emmanuel1514 committed 3 days ago	2cc1666	
Merge pull request #76 from alinsnhz/59-adaptar-unocs-para-usar-el-estado-recibido-por-sockets Emmanuel1514 authored 3 days ago	Verified	2425ed9
Merge branch 'main' into 59-adaptar-unocs-para-usar-el-estado-recibido-por-sockets Emmanuel1514 authored 3 days ago	Verified	47557d8
Se agrego CartaCentro para evitar errores Emmanuel1514 committed 3 days ago	1d9c7e7	
Correccion de errores en su mayoria por el nombre Emmanuel1514 committed 3 days ago	61f5709	
modifica servidor para los sockets alinanhz committed 3 days ago	ce98b1a	
crea clase servidor para API alinanhz committed 3 days ago	d3edb7c	
agrega endpoint para cartas robadas alinanhz committed 3 days ago	8248a15	
modifica conexion con la BD alinanhz committed 3 days ago	f2ab2c1	
cambia json y program alinanhz committed 3 days ago	8468432	
resuelve errores alinanhz committed 4 days ago	cda5bbf	
cambios alinanhz committed 4 days ago	e37b5c1	
resuleve conflictos pt.4 alinanhz committed 4 days ago	4e2ffd3	
resuelve conflictos pt.3 alinanhz committed 4 days ago	aae1a57	
resuelve conflictos pt.2 alinanhz committed 4 days ago	2debe44	
resuelve errores de conflictos alinanhz committed 4 days ago	9339897	

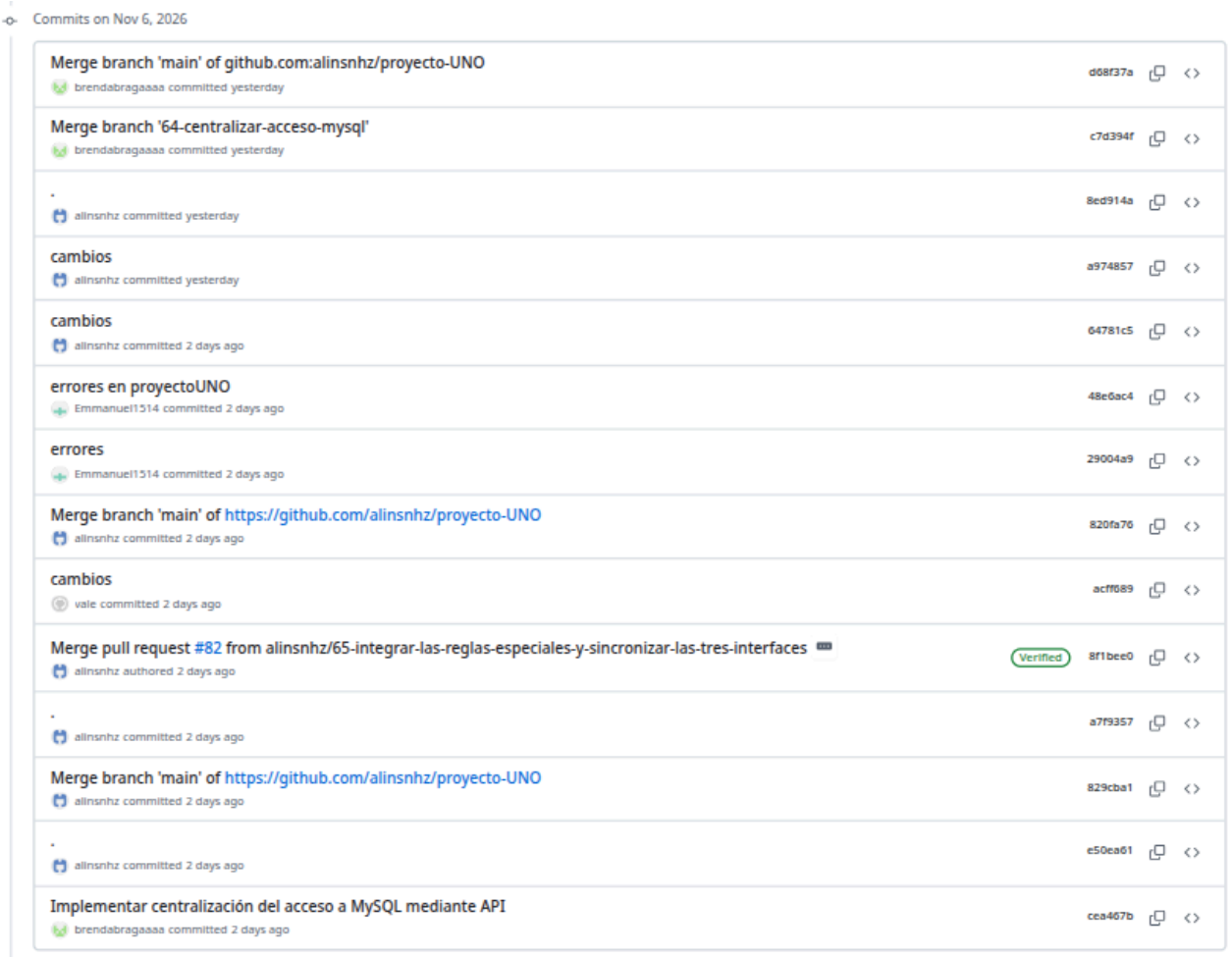
05/11/2026

Commits on Nov 5, 2026

Merge pull request #79 from alinsnhz/61-enviar-las-acciones-de-juego-desde-los-controles-de-windows-forms Emmanuel1514 authored 2 days ago	Verified	f5e5ab7
Se Completa el Issure Conectar los botones de la interfaz con el protocolo de mensajes para que las acciones se procesen en el servidor. Emmanuel1514 committed 2 days ago	ec61162	
Actualiza servicio de comunicación con API brendabragaaaa committed 2 days ago	567149d	
Se solucionaron errores Emmanuel1514 committed 2 days ago	5781ce6	
Validaciones y endpoints de partida brendabragaaaa committed 2 days ago	8bb02c6	


Merge pull request #81 from alinsnhz/65-integrar-las-reglas-especiales-y-sincronizar-las-tres-interfaces
alinsnhz authored 2 days ago
Verified
f422762
agrega comunicación entre cliente-servidor
alinsnhz committed 2 days ago
0c87057
Merge pull request #80 from alinsnhz/57-enviar-estados-privados-y-públicos-de-la-partida
Emmanuel1514 authored 2 days ago
Verified
6d001fc
se hace issure solo se Evita que dos acciones se procesen y se notifiquen al mismo tiempo.
Emmanuel1514 committed 2 days ago
649caa6
.
alinsnhz committed 2 days ago
843e57b
agrega reglas al servidor
alinsnhz committed 2 days ago
0a7182b
Merge branch '63-completar-endpoints'
brendabragaaaa committed 2 days ago
def0604
Completa registro de jugadas y cartas de penalización
brendabragaaaa committed 2 days ago
6e078e3
Integra servidor TCP con API y registro de partida
brendabragaaaa committed 2 days ago
e5b9ac4

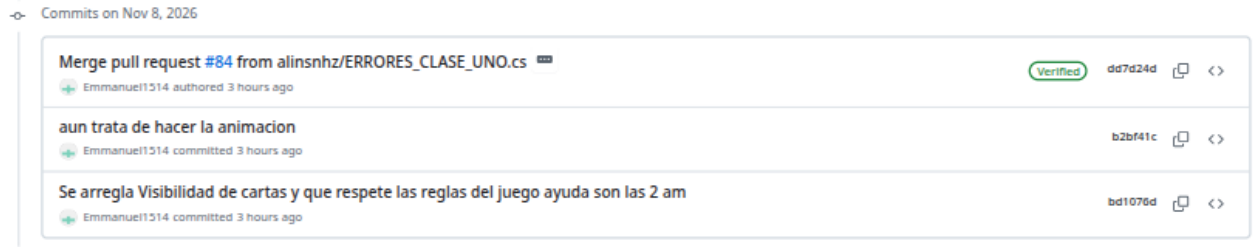
06/11/2026


Commits on Nov 6, 2026
Merge branch 'main' of github.com:alinsnhz/proyecto-UNO
brendabragaaaa committed yesterday
d68f37a
Merge branch '64-centralizar-acceso-mysql'
brendabragaaaa committed yesterday
c7d394f
.
alinsnhz committed yesterday
8ed914a
cambios
alinsnhz committed yesterday
a974857
cambios
alinsnhz committed 2 days ago
64781c5
errores en proyectoUNO
Emmanuel1514 committed 2 days ago
48e6ac4
errores
Emmanuel1514 committed 2 days ago
29004a9
Merge branch 'main' of https://github.com/alinsnhz/proyecto-UNO
alinsnhz committed 2 days ago
820fa76
cambios
vale committed 2 days ago
acff689
Merge pull request #82 from alinsnhz/65-integrar-las-reglas-especiales-y-sincronizar-las-tres-interfaces
alinsnhz authored 2 days ago
Verified
8f1bee0
.
alinsnhz committed 2 days ago
a7f9357
Merge branch 'main' of https://github.com/alinsnhz/proyecto-UNO
alinsnhz committed 2 days ago
829cba1
.
alinsnhz committed 2 days ago
e50ea61
Implementar centralización del acceso a MySQL mediante API
brendabragaaaa committed 2 days ago
cea467b

07/11/2026



08/11/2026



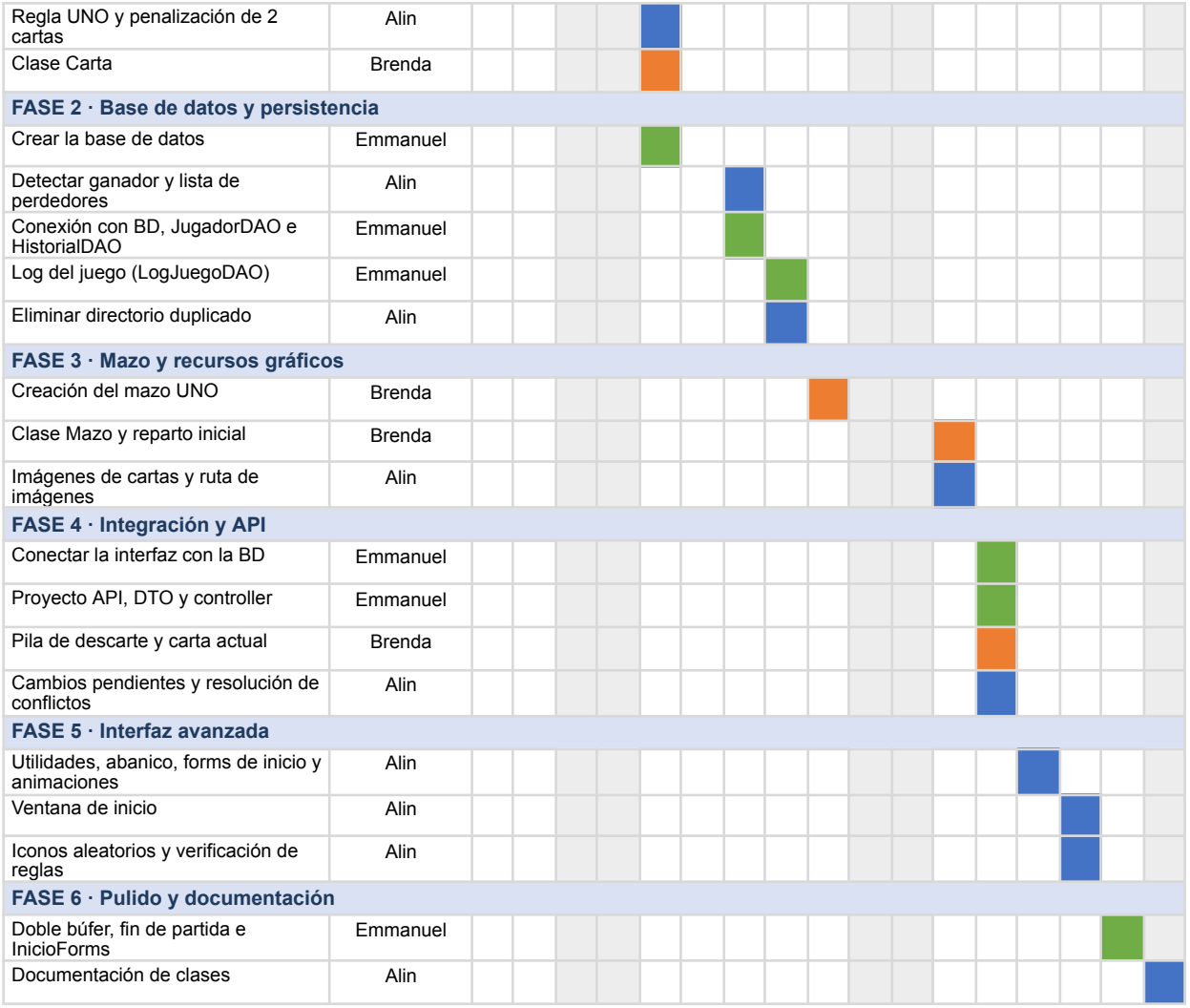
- Cronograma de actividades

El cronograma se elaboró a partir del listado de commits del repositorio, del 24 de septiembre al 10 de octubre de 2026. Cada barra indica los días en que se trabajó la actividad y su color corresponde al responsable. Las actividades del 30 de septiembre se agruparon en ese día porque los commits se registraron en horas cercanas.

Responsables: ■ Alin ■ Emmanuel ■ Brenda ■ Compartida ▒ Fin de semana

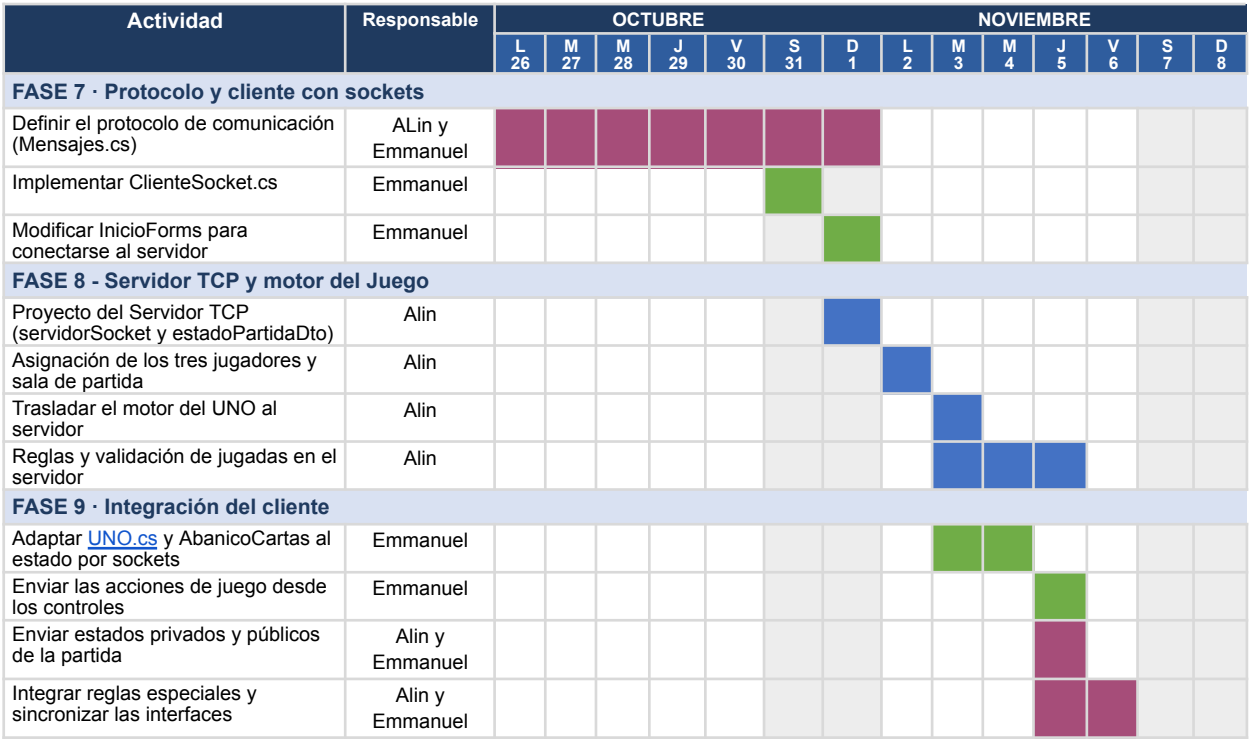
Primera etapa (24 de septiembre al 10 de octubre)

Actividad	Responsable	SEPTIEMBRE							OCTUBRE									
		J 24	V 25	S 26	D 27	L 28	M 29	M 30	J 1	V 2	S 3	D 4	L 5	M 6	M 7	J 8	V 9	S 10
FASE 1 · Lógica base e interfaz inicial																		
Commit inicial del repositorio	Emmanuel	■																
Estructura del proyecto y clase Jugador	Alin		■															
Clase juegoUNO y validación de cartas	Alin			■														
Interfaz principal y cartas como botones	Emmanuel			■	■	■												
Cartas especiales (reversa, salto, +2, +4)	Alin				■													



Todas las actividades se encuentran completadas al 10 de octubre de 2026.

Segunda etapa: cliente-servidor (26 de octubre al 07 de noviembre)



FASE 10 · API y base de datos/ Reporte															
Configurar la API para ejecutarse en la red	Alin														
Completar los endpoints de partidas, eventos y resultados	Brenda														
Centralizar el acceso a MySQL mediante la API	Brenda														
Integrar el servidor TCP con la API y MySQL	Brenda y Emmanuel														
Actualización y Finalización del Reporte	Alin, Brenda y Emmanuel														

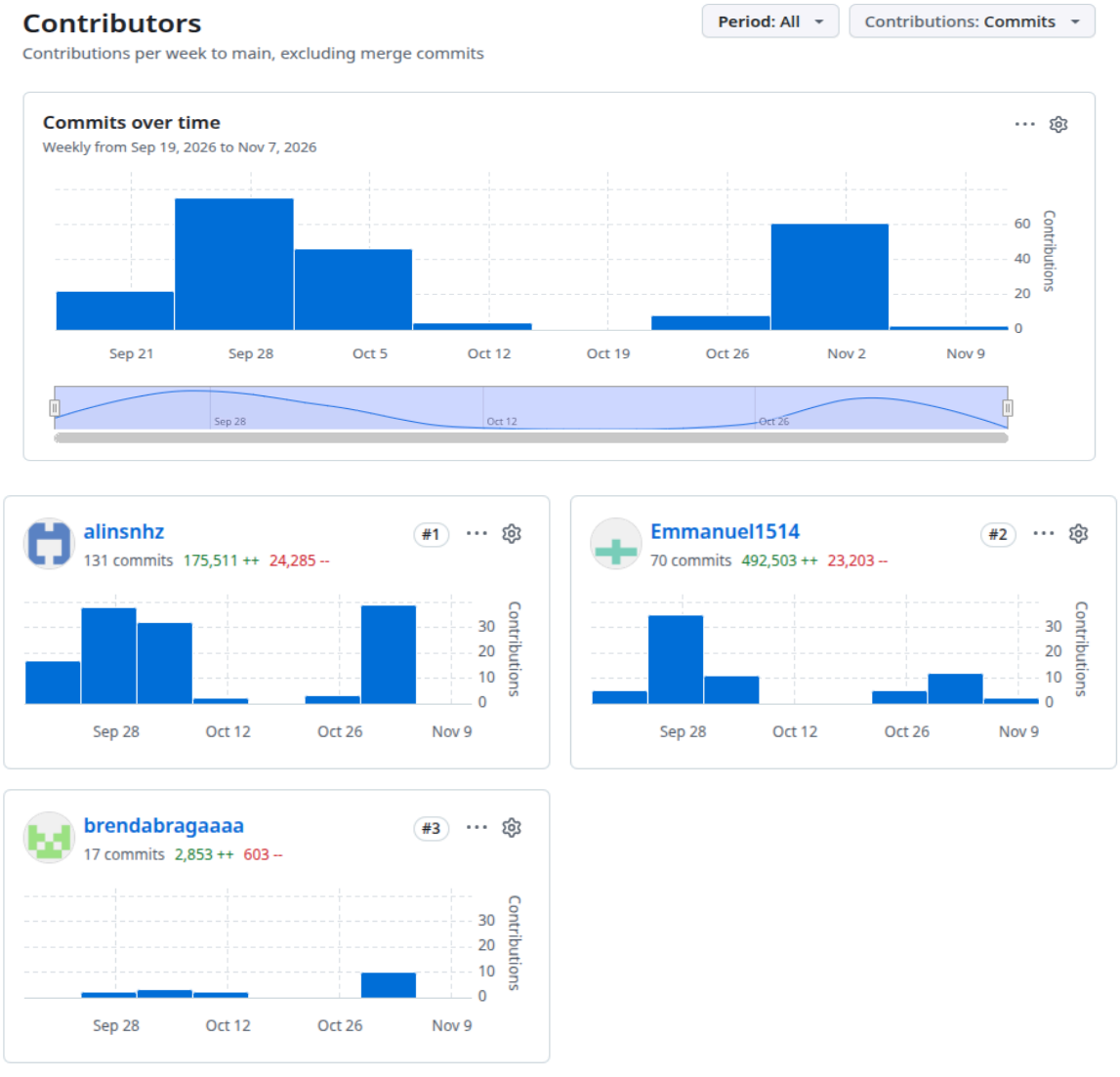
Todas las actividades se encuentran completadas al 08 de noviembre de 2026.

- Resumen

RESUMEN POR RESPONSABLE

Responsable	Cuenta GitHub	Actividades	Días de trabajo	Commits (a main)
Alin	alinsnhz	19	28	131
Emmanuel	Emmanuel1514	18	28	71
Brenda	brendabragaaaa	10	15	17
Total		**47**	**71**	**219**

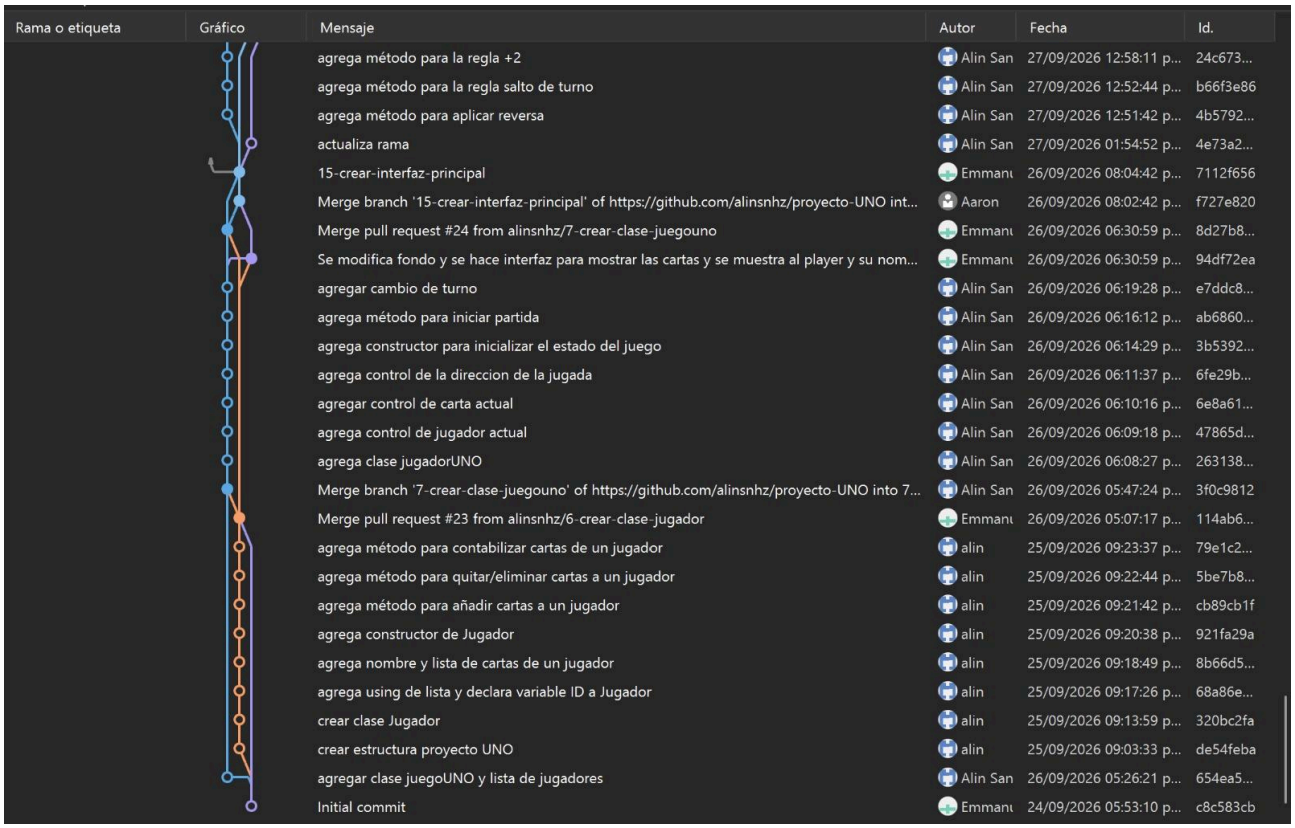
- Gráfica de commits

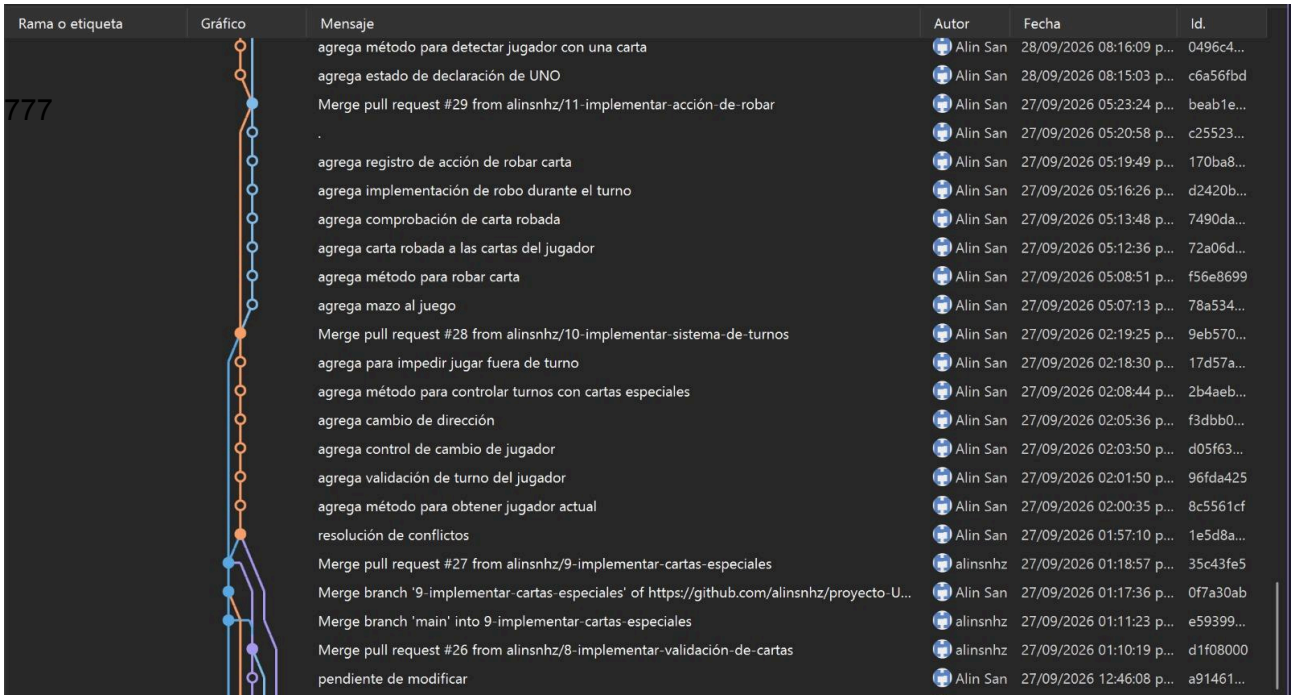


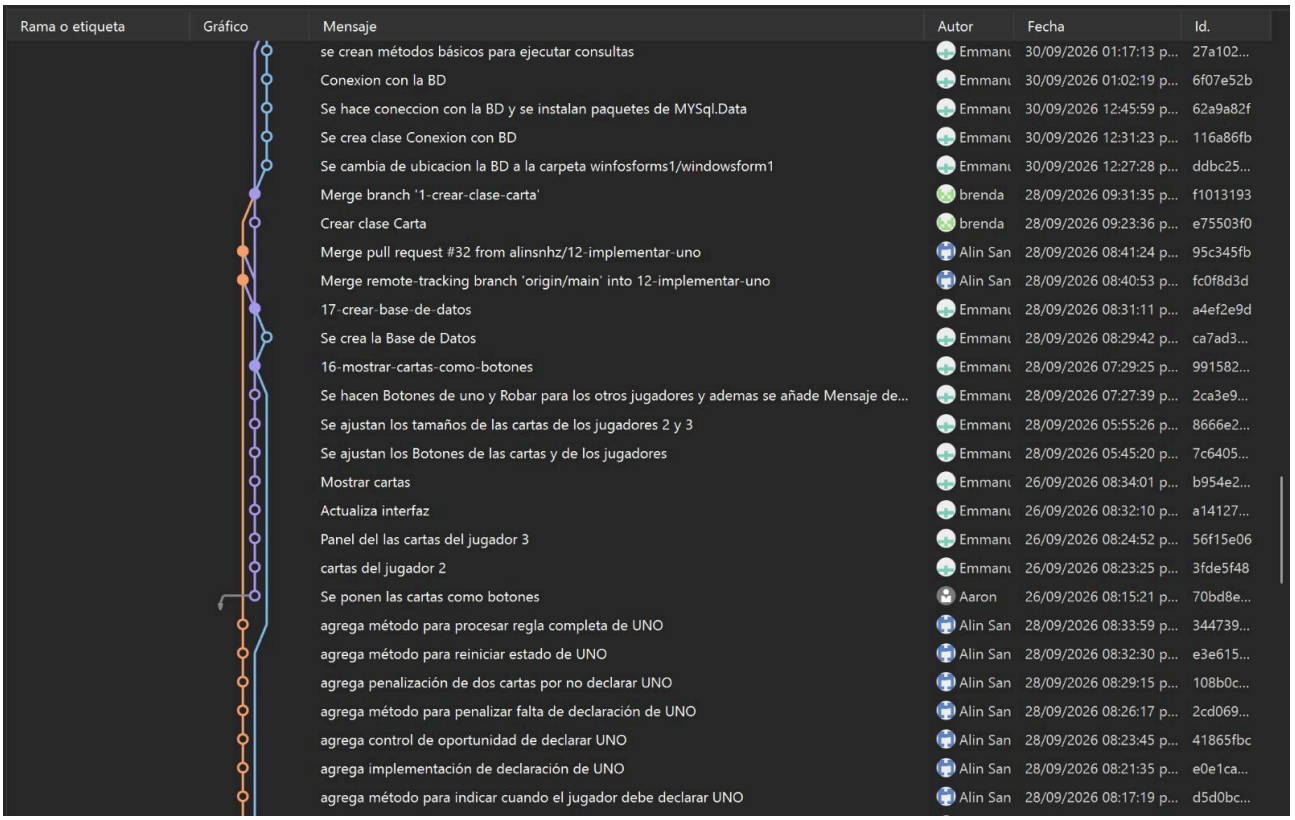
- Listado de ramas generadas

La evolución de las *branches* del proyecto se documentó mediante la vista de gráficos de Git en Visual Studio, que permite visualizar el historial de *commits*, las ramas creadas y las integraciones realizadas a lo largo del desarrollo. En esta vista se pueden identificar las distintas líneas de trabajo, los cambios registrados por cada integrante y los *merges* mediante los cuales las modificaciones de las ramas secundarias se incorporaron a la rama principal. Cada rama se identifica por su nombre, generalmente relacionado con la funcionalidad desarrollada o la *issue* correspondiente, mientras que los mensajes de los *commits* permiten conocer los cambios realizados en cada etapa. De esta manera, la representación gráfica facilita la comprensión de cómo evolucionó el proyecto, cómo se distribuyeron las tareas entre los integrantes y cómo se integraron progresivamente las funcionalidades hasta llegar a la versión final.

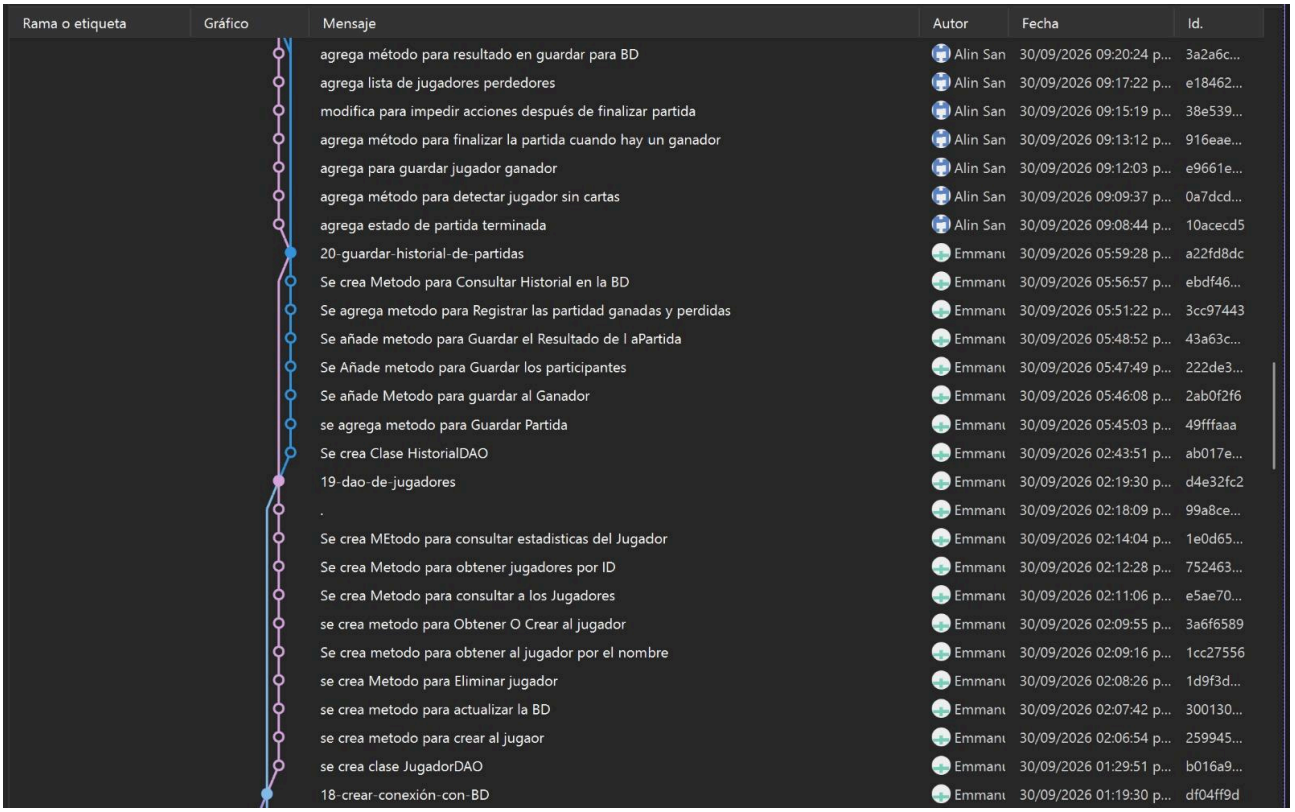
Primera Etapa



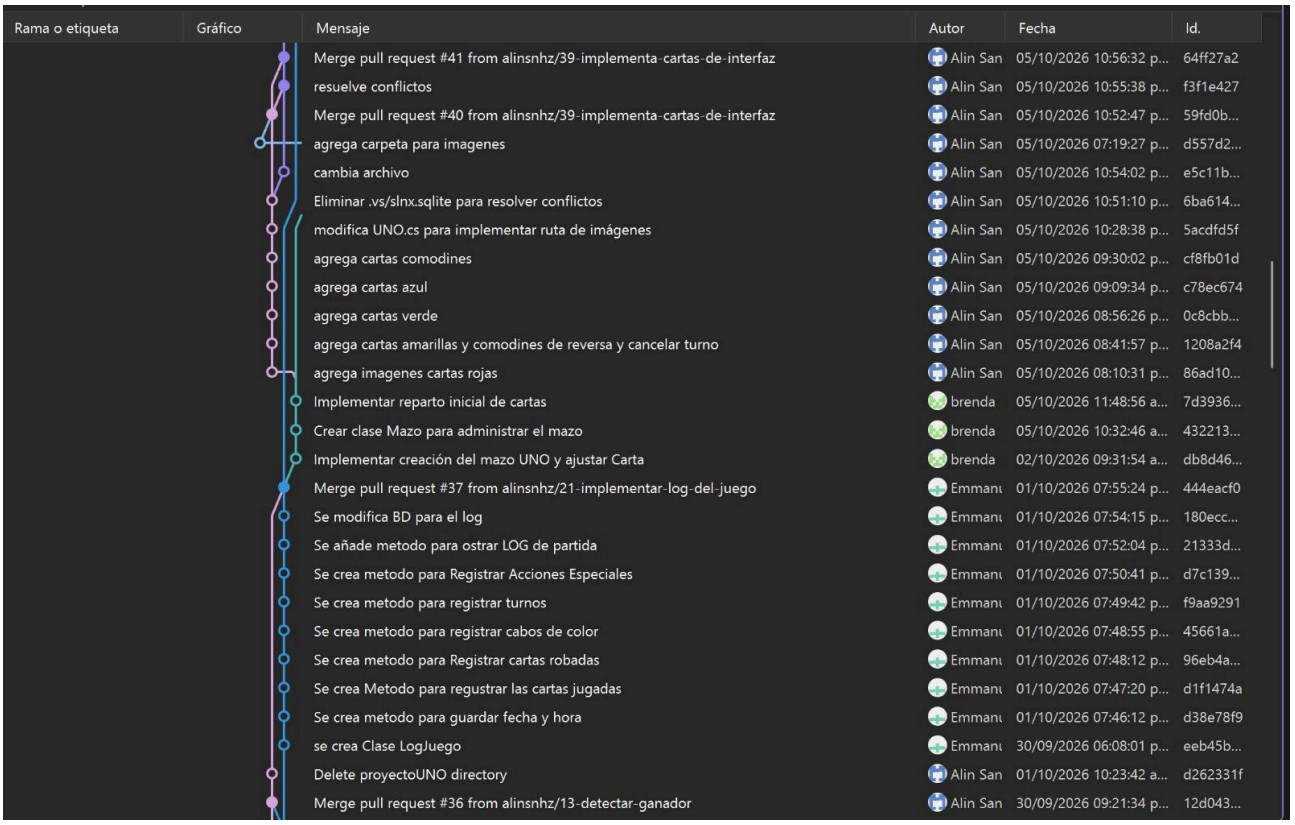




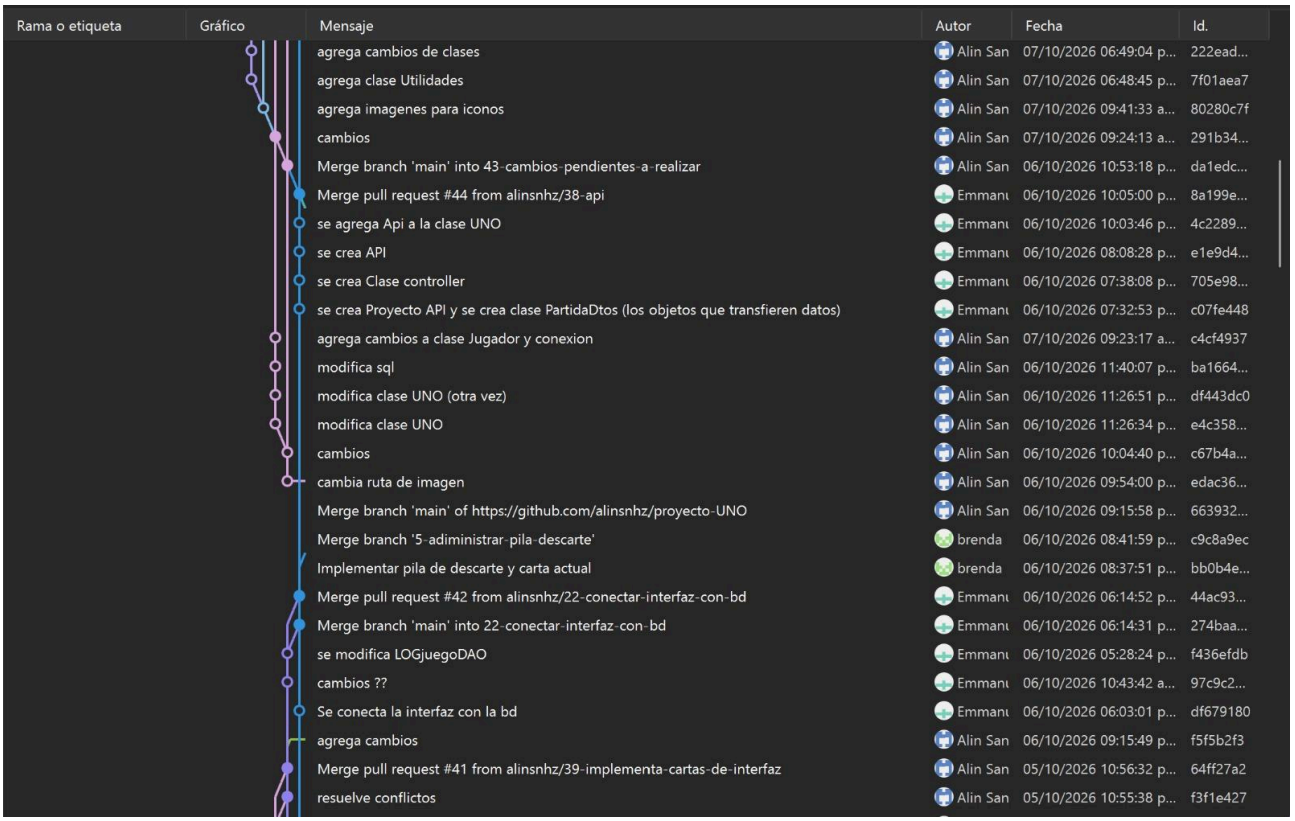
Rama o etiqueta	Gráfico	Mensaje	Autor	Fecha	Id.
		se crean métodos básicos para ejecutar consultas	Emmanı	30/09/2026 01:17:13 p...	27a102...
		Conexion con la BD	Emmanı	30/09/2026 01:02:19 p...	6f07e52b
		Se hace coneccion con la BD y se instalan paquetes de MYSql.Data	Emmanı	30/09/2026 12:45:59 p...	62a9a82f
		Se crea clase Conexion con BD	Emmanı	30/09/2026 12:31:23 p...	116a86fb
		Se cambia de ubicacion la BD a la carpeta winfosforms1/windowsform1	Emmanı	30/09/2026 12:27:28 p...	ddbc25...
		Merge branch '1-crear-clase-carta'	brenda	28/09/2026 09:31:35 p...	f1013193
		Crear clase Carta	brenda	28/09/2026 09:23:36 p...	e75503f0
		Merge pull request #32 from alinsnhz/12-implementar-uno	Alin San	28/09/2026 08:41:24 p...	95c345fb
		Merge remote-tracking branch 'origin/main' into 12-implementar-uno	Alin San	28/09/2026 08:40:53 p...	fc0f8d3d
		17-crear-base-de-datos	Emmanı	28/09/2026 08:31:11 p...	a4ef2e9d
		Se crea la Base de Datos	Emmanı	28/09/2026 08:29:42 p...	ca7ad3...
		16-mostrar-cartas-como-botones	Emmanı	28/09/2026 07:29:25 p...	991582...
		Se hacen Botones de uno y Robar para los otros jugadores y ademas se añade Mensaje de...	Emmanı	28/09/2026 07:27:39 p...	2ca3e9...
		Se ajustan los tamaños de las cartas de los jugadores 2 y 3	Emmanı	28/09/2026 05:55:26 p...	8666e2...
		Se ajustan los Botones de las cartas y de los jugadores	Emmanı	28/09/2026 05:45:20 p...	7c6405...
		Mostrar cartas	Emmanı	26/09/2026 08:34:01 p...	b954e2...
		Actualiza interfaz	Emmanı	26/09/2026 08:32:10 p...	a14127...
		Panel del las cartas del jugador 3	Emmanı	26/09/2026 08:24:52 p...	56f15e06
		cartas del jugador 2	Emmanı	26/09/2026 08:23:25 p...	3fde5f48
		Se ponen las cartas como botones	Aaron	26/09/2026 08:15:21 p...	70bd8e...
		agrega método para procesar regla completa de UNO	Alin San	28/09/2026 08:33:59 p...	344739...
		agrega método para reiniciar estado de UNO	Alin San	28/09/2026 08:32:30 p...	e3e615...
		agrega penalización de dos cartas por no declarar UNO	Alin San	28/09/2026 08:29:15 p...	108b0c...
		agrega método para penalizar falta de declaración de UNO	Alin San	28/09/2026 08:26:17 p...	2cd069...
		agrega control de oportunidad de declarar UNO	Alin San	28/09/2026 08:23:45 p...	41865fbc
		agrega implementación de declaración de UNO	Alin San	28/09/2026 08:21:35 p...	e0e1ca...
		agrega método para indicar cuando el jugador debe declarar UNO	Alin San	28/09/2026 08:17:19 p...	d5d0bc...



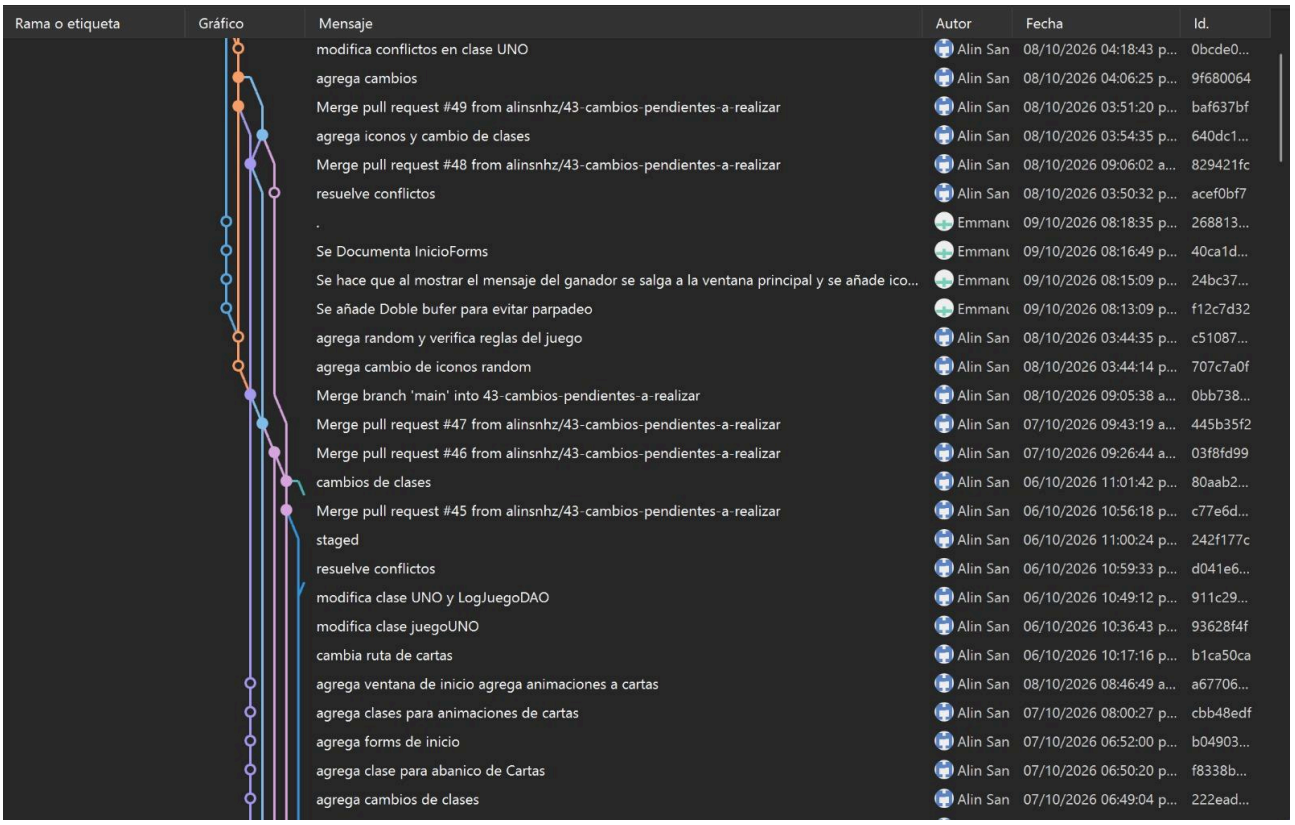
Rama o etiqueta	Gráfico	Mensaje	Autor	Fecha	Id.
		agrega método para resultado en guardar para BD	Alin San	30/09/2026 09:20:24 p...	3a2a6c...
		agrega lista de jugadores perdedores	Alin San	30/09/2026 09:17:22 p...	e18462...
		modifica para impedir acciones después de finalizar partida	Alin San	30/09/2026 09:15:19 p...	38e539...
		agrega método para finalizar la partida cuando hay un ganador	Alin San	30/09/2026 09:13:12 p...	916eae...
		agrega para guardar jugador ganador	Alin San	30/09/2026 09:12:03 p...	e9661e...
		agrega método para detectar jugador sin cartas	Alin San	30/09/2026 09:09:37 p...	0a7dcd...
		agrega estado de partida terminada	Alin San	30/09/2026 09:08:44 p...	10acecd5
		20-guardar-historial-de-partidas	Emmanı	30/09/2026 05:59:28 p...	a22fd8dc
		Se crea Metodo para Consultar Historial en la BD	Emmanı	30/09/2026 05:56:57 p...	ebdf46...
		Se agrega metodo para Registrar las partidad ganadas y perdidas	Emmanı	30/09/2026 05:51:22 p...	3cc97443
		Se añade metodo para Guardar el Resultado de l aPartida	Emmanı	30/09/2026 05:48:52 p...	43a63c...
		Se Añade metodo para Guardar los participantes	Emmanı	30/09/2026 05:47:49 p...	222de3...
		Se añade Metodo para guardar al Ganador	Emmanı	30/09/2026 05:46:08 p...	2ab0f2f6
		se agrega metodo para Guardar Partida	Emmanı	30/09/2026 05:45:03 p...	49fffaaa
		Se crea Clase HistorialDAO	Emmanı	30/09/2026 02:43:51 p...	ab017e...
		19-dao-de-jugadores	Emmanı	30/09/2026 02:19:30 p...	d4e32fc2
		.	Emmanı	30/09/2026 02:18:09 p...	99a8ce...
		Se crea MEtodo para consultar estadisticas del Jugador	Emmanı	30/09/2026 02:14:04 p...	1e0d65...
		Se crea Metodo para obtener jugadores por ID	Emmanı	30/09/2026 02:12:28 p...	752463...
		Se crea Metodo para consultar a los Jugadores	Emmanı	30/09/2026 02:11:06 p...	e5ae70...
		se crea metodo para Obtener O Crear al jugador	Emmanı	30/09/2026 02:09:55 p...	3a6f6589
		Se crea metodo para obtener al jugador por el nombre	Emmanı	30/09/2026 02:09:16 p...	1cc27556
		se crea Metodo para Eliminar jugador	Emmanı	30/09/2026 02:08:26 p...	1d9f3d...
		se crea metodo para actualizar la BD	Emmanı	30/09/2026 02:07:42 p...	300130...
		se crea metodo para crear al jugaor	Emmanı	30/09/2026 02:06:54 p...	259945...
		se crea clase JugadorDAO	Emmanı	30/09/2026 01:29:51 p...	b016a9...
		18-crear-conexión-con-BD	Emmanı	30/09/2026 01:19:30 p...	df04ff9d

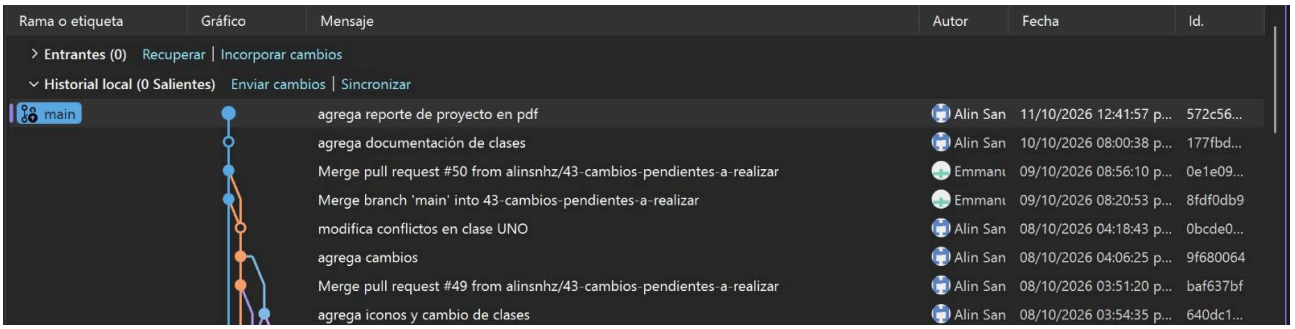


Rama o etiqueta	Gráfico	Mensaje	Autor	Fecha	Id.
		Merge pull request #41 from alinsnhz/39-implementa-cartas-de-interfaz	Alin San	05/10/2026 10:56:32 p...	64ff27a2
		resuelve conflictos	Alin San	05/10/2026 10:55:38 p...	f3f1e427
		Merge pull request #40 from alinsnhz/39-implementa-cartas-de-interfaz	Alin San	05/10/2026 10:52:47 p...	59fd0b...
		agrega carpeta para imagenes	Alin San	05/10/2026 07:19:27 p...	d557d2...
		cambia archivo	Alin San	05/10/2026 10:54:02 p...	e5c11b...
		Eliminar .vs/slnx.sqlite para resolver conflictos	Alin San	05/10/2026 10:51:10 p...	6ba614...
		modifica UNO.cs para implementar ruta de imágenes	Alin San	05/10/2026 10:28:38 p...	5acdfd5f
		agrega cartas comodines	Alin San	05/10/2026 09:30:02 p...	cf8fb01d
		agrega cartas azul	Alin San	05/10/2026 09:09:34 p...	c78ec674
		agrega cartas verde	Alin San	05/10/2026 08:56:26 p...	0c8cbb...
		agrega cartas amarillas y comodines de reversa y cancelar turno	Alin San	05/10/2026 08:41:57 p...	1208a2f4
		agrega imagenes cartas rojas	Alin San	05/10/2026 08:10:31 p...	86ad10...
		Implementar reparto inicial de cartas	brenda	05/10/2026 11:48:56 a...	7d3936...
		Crear clase Mazo para administrar el mazo	brenda	05/10/2026 10:32:46 a...	432213...
		Implementar creación del mazo UNO y ajustar Carta	brenda	02/10/2026 09:31:54 a...	db8d46...
		Merge pull request #37 from alinsnhz/21-implementar-log-del-juego	Emmanı	01/10/2026 07:55:24 p...	444eacf0
		Se modifica BD para el log	Emmanı	01/10/2026 07:54:15 p...	180ecc...
		Se añade metodo para ostrar LOG de partida	Emmanı	01/10/2026 07:52:04 p...	21333d...
		Se crea metodo para Registrar Acciones Especiales	Emmanı	01/10/2026 07:50:41 p...	d7c139...
		Se crea metodo para registrar turnos	Emmanı	01/10/2026 07:49:42 p...	f9aa9291
		Se crea metodo para registrar cabos de color	Emmanı	01/10/2026 07:48:55 p...	45661a...
		Se crea metodo para Registrar cartas robadas	Emmanı	01/10/2026 07:48:12 p...	96eb4a...
		Se crea Metodo para regustrar las cartas jugadas	Emmanı	01/10/2026 07:47:20 p...	d1f1474a
		Se crea metodo para guardar fecha y hora	Emmanı	01/10/2026 07:46:12 p...	d38e78f9
		se crea Clase LogJuego	Emmanı	30/09/2026 06:08:01 p...	eeb45b...
		Delete proyectoUNO directory	Alin San	01/10/2026 10:23:42 a...	d262331f
		Merge pull request #36 from alinsnhz/13-detectar-ganador	Alin San	30/09/2026 09:21:34 p...	12d043...



Rama o etiqueta	Gráfico	Mensaje	Autor	Fecha	Id.
		agrega cambios de clases	Alin San	07/10/2026 06:49:04 p...	222ead...
		agrega clase Utilidades	Alin San	07/10/2026 06:48:45 p...	7f01aea7
		agrega imagenes para iconos	Alin San	07/10/2026 09:41:33 a...	80280c7f
		cambios	Alin San	07/10/2026 09:24:13 a...	291b34...
		Merge branch 'main' into 43-cambios-pendientes-a-realizar	Alin San	06/10/2026 10:53:18 p...	da1edc...
		Merge pull request #44 from alinsnhz/38-api	Emmanı	06/10/2026 10:05:00 p...	8a199e...
		se agrega Api a la clase UNO	Emmanı	06/10/2026 10:03:46 p...	4c2289...
		se crea API	Emmanı	06/10/2026 08:08:28 p...	e1e9d4...
		se crea Clase controller	Emmanı	06/10/2026 07:38:08 p...	705e98...
		se crea Proyecto API y se crea clase PartidaDtos (los objetos que transfieren datos)	Emmanı	06/10/2026 07:32:53 p...	c07fe448
		agrega cambios a clase Jugador y conexion	Alin San	07/10/2026 09:23:17 a...	c4cf4937
		modifica sql	Alin San	06/10/2026 11:40:07 p...	ba1664...
		modifica clase UNO (otra vez)	Alin San	06/10/2026 11:26:51 p...	df443dc0
		modifica clase UNO	Alin San	06/10/2026 11:26:34 p...	e4c358...
		cambios	Alin San	06/10/2026 10:04:40 p...	c67b4a...
		cambia ruta de imagen	Alin San	06/10/2026 09:54:00 p...	edac36...
		Merge branch 'main' of https://github.com/alinsnhz/proyecto-UNO	Alin San	06/10/2026 09:15:58 p...	663932...
		Merge branch '5-adiministrar-pila-descarte'	brenda	06/10/2026 08:41:59 p...	c9c8a9ec
		Implementar pila de descarte y carta actual	brenda	06/10/2026 08:37:51 p...	bb0b4e...
		Merge pull request #42 from alinsnhz/22-conectar-interfaz-con-bd	Emmanı	06/10/2026 06:14:52 p...	44ac93...
		Merge branch 'main' into 22-conectar-interfaz-con-bd	Emmanı	06/10/2026 06:14:31 p...	274baa...
		se modifica LOGjuegoDAO	Emmanı	06/10/2026 05:28:24 p...	f436efdb
		cambios ??	Emmanı	06/10/2026 10:43:42 a...	97c9c2...
		Se conecta la interfaz con la bd	Emmanı	06/10/2026 06:03:01 p...	df679180
		agrega cambios	Alin San	06/10/2026 09:15:49 p...	f5f5b2f3
		Merge pull request #41 from alinsnhz/39-implementa-cartas-de-interfaz	Alin San	05/10/2026 10:56:32 p...	64ff27a2
		resuelve conflictos	Alin San	05/10/2026 10:55:38 p...	f3f1e427





Segunda Etapa

Issue	Rama (tarea)	Responsable	Pull request
#52	Definir el protocolo de comunicación entre cliente y servidor	Alin y Emmanuel	#71
#53	Implementar ClienteSocket.cs en Windows Forms	Emmanuel	#70
#54	Asignación de los tres jugadores y sala de partida	Alin	#73
#55	Trasladar el motor del UNO al servidor	Alin	#74, #75
#57	Enviar estados privados y públicos de la partida	Alin y Emmanuel	#80
#58	Modificar InicioForms.cs para conectarse al servidor	Emmanuel	#72
#59	Adaptar UNO.cs para usar el estado recibido por sockets	Emmanuel	#76
#60	Adaptar AbanicoCartas.cs para mostrar únicamente la mano propia	Emmanuel	—
#61	Enviar las acciones de juego desde los controles	Emmanuel	#79
#62	Configurar la API para ejecutarse en la red	Alin	#78

Issue	Rama (tarea)	Responsable	Pull request
#63	Completar los endpoints de partidas, eventos y resultados	Brenda	—
#64	Centralizar el acceso a MySQL	Brenda	—
#65	Integrar las reglas especiales y sincronizar las tres interfaces	Alin y Emmanuel	#81, #82
#66	Integrar el servidor TCP con la API y MySQL	Brenda y Emmanuel	—

- Problemas encontrados y cómo los solucionaron

1. Conflictos de fusión (*Merge Conflicts*) en Git

- **Problema:** Al trabajar de forma paralela en las clases lógicas principales (UNO.cs, JuegoUNO.cs) y la capa de datos, se generaron conflictos al hacer *Merge Pull Request* en las ramas de integración.
- **Solución:** Se establecieron reglas de sincronización previa (git pull origin main) antes de subir *commits*, y se resolvieron manualmente los conflictos revisando el código línea por línea en VS Code/Visual Studio antes de aprobar las solicitudes de extracción

2. Parpadeo visual (*Flicker*) y renderizado lento de cartas en la Interfaz Gráfica

- **Problema:** Al actualizar la mano del jugador o ejecutar animaciones de movimiento de cartas, la pantalla mostraba un parpadeo constante (*flicker*) debido al redibujado de los controles nativos de Windows Forms.
- **Solución:** Se habilitó la propiedad DoubleBuffered = true en el formulario principal y en los contenedores de las cartas, además de optimizar la carga de imágenes usando recursos almacenados en memoria local en lugar de cargarlos directamente desde el disco en cada turno.

3. Manejo de estados especiales y penalización de la regla "UNO"

- **Problema:** El juego no detectaba con precisión el momento exacto en que un jugador omitía presionar el botón "UNO" antes de pasar el turno, provocando que la penalización de 2 cartas no se aplicara a tiempo o se aplicara al jugador equivocado.
- **Solución:** Se implementó una bandera de estado (*boolean flag*) dentro del controlador de turnos (JuegoUNO.cs) que evalúa si el jugador tiene 1 sola carta al momento de finalizar su jugada. Si la declaración de UNO no fue activada en ese lapso, el sistema aplica automáticamente el método de penalización para robar dos cartas e incluye el registro en el log.

4. Persistencia asíncrona del Log de Juego en la Base de Datos

- **Problema:** Guardar cada movimiento (cartas robadas, jugadas, cambios de color) de manera síncrona producía pequeñas pausas o congelamientos en la interfaz de usuario durante la partida.
- **Solución:** Se reestructuraron las consultas del DAO (LogJuegoDAO) y los llamados a la API para que las operaciones de inserción en la base de datos SQLite/MySQL se ejecuten

de fondo en segundo plano (*Background Worker / Async Tasks*), manteniendo la interfaz fluida.

5. Sincronización de partidas y prevención de registros duplicados en la Base de Datos

- **Problema:** Durante la integración del servidor TCP con la API y la base de datos MySQL, surgieron dificultades para mantener correctamente asociados los identificadores de las partidas y de los jugadores. Además, era necesario evitar que una misma partida o su resultado final se registraran más de una vez, especialmente cuando se procesaban varias acciones o se interrumpía la conexión de un jugador.

- **Solución:** Se implementó un mecanismo de registro y seguimiento de las partidas desde el servidor, almacenando los identificadores devueltos por la API y asociándolos con los jugadores conectados. También se incorporaron controles para evitar registros duplicados y garantizar que la finalización de la partida y sus resultados se guardaran una sola vez. Para mantener la consistencia de los datos, se utilizaron transacciones en las operaciones de persistencia del DAO.

6. Sincronización de los tres clientes y privacidad de las manos

- **Problema:** Al pasar a una arquitectura cliente-servidor, las tres interfaces debian mostrar la misma partida sin que un jugador viera las cartas de los demas

- **Solución:** El servidor mantiene un único estado de la partida y envía a todos los clientes el estado público y a cada jugador su estado privado. El cliente dibuja la mesa con el estado recibido por sockets y AbanicoCartas muestra únicamente la mano propia.

7. Incompatibilidad de la versión de .NET

- **Problema:** Al ejecutar el proyecto en distintos equipos aparecieron errores por la versión de .NET instalada, y la aplicación no se iniciaba correctamente.

- **Solución:** Se instaló el runtime dotnet-runtime-10.0.14-win-x64 en los equipos, con lo que la aplicación y el servidor pudieron ejecutarse sin errores de versión.

8. Fallos por el nombre del jugador

- **Problema:** El servidor registra a cada jugador con el nombre que envía el cliente en su mensaje de conexión. Las conexiones fallaban repetidamente porque el nombre del jugador no se escribía como el sistema lo esperaba, y durante un tiempo el equipo no pudo identificar la causa del error.

- **Solución:** Con la ayuda del profesor se encontró el origen del fallo y se corrigió; en el historial de commits aparece la corrección de errores que provenían en su mayoría del nombre. Desde entonces el nombre se captura y se envía al servidor como corresponde.

Capturas de la aplicación:

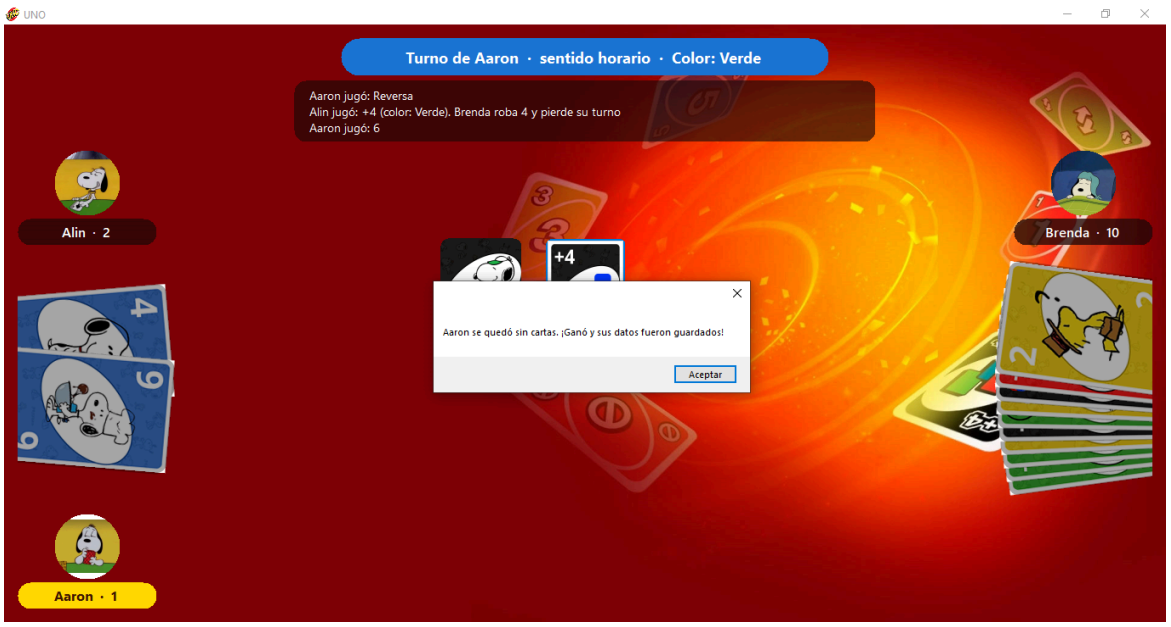
En esta sección se muestran las pantallas principales del juego en funcionamiento



Ventana de inicio.



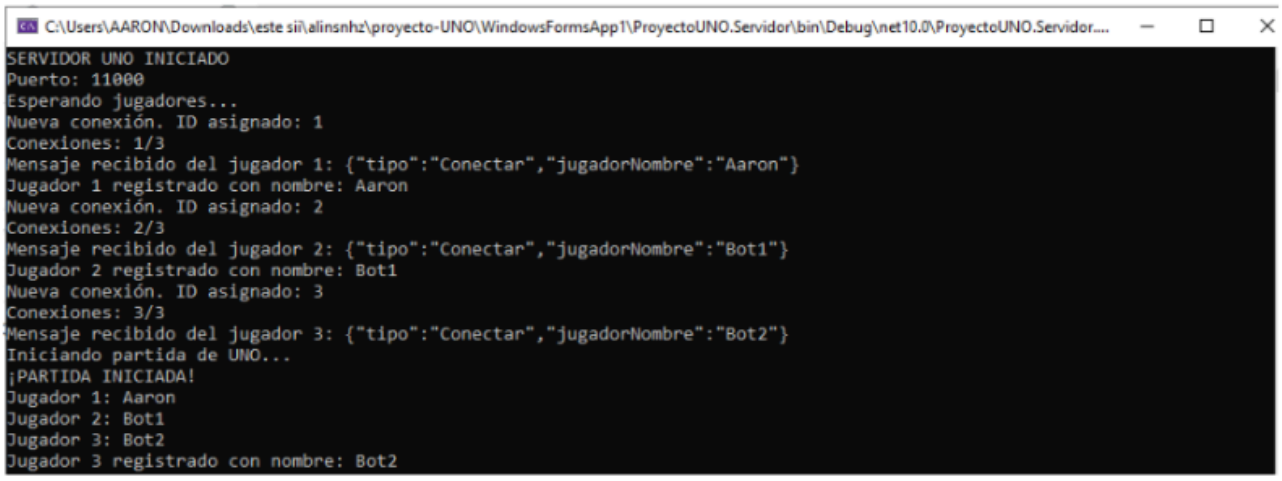
Partida en curso con los tres jugadores.



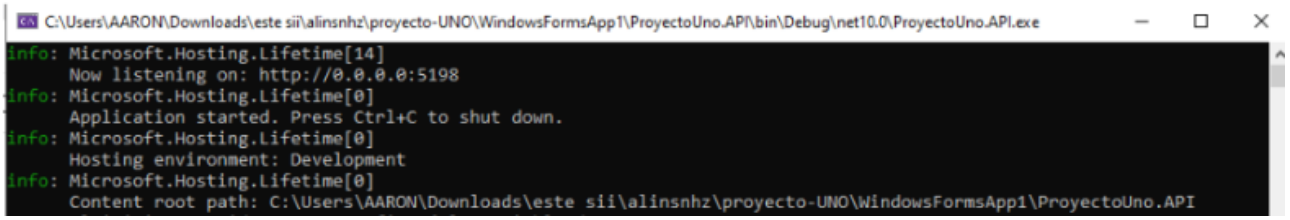
Mensaje del ganador.



Cliente conectado a la partida (jugador Aaron, con Bot1 y Bot2 como rivales).



Servidor TCP en ejecución: puerto 11000, conexiones de los tres jugadores e inicio de la partida.



API en ejecución, escuchando en http://0.0.0.0:5198.

- Conclusión personal

Martinez Sanchez Mayra Alin

Este proyecto me permitió aplicar lo visto en la materia: crear una interfaz gráfica en C# y conectarla con una base de datos y una API, organizando todo en capas. Me encargué sobre todo de la lógica del juego (turnos, validación de cartas, cartas especiales, regla UNO y detección del ganador) y de integrarla con la interfaz. Lo más difícil fue lograr que todo funcionara junto, en especial los efectos combinados de +2 y +4 y la penalización por no declarar UNO.

En la segunda etapa trasladé el motor del juego a un servidor TCP independiente: creé el proyecto del servidor, implementé la asignación de los tres jugadores y la sala de partida, la validación de reglas y jugadas en el servidor y la configuración de la API para ejecutarse en la red. También definí con Emmanuel el protocolo de comunicación y el envío de estados privados y públicos.

También aprendí a trabajar en equipo con Git y GitHub: ramas por tarea, pull requests y resolución de conflictos. Si lo repitiera, haría las pruebas de integración antes y repartiría el trabajo de forma más equilibrada. Aun así, el resultado cumple con lo pedido: un juego funcional con historial de partidas y log de movimientos.

Rodriguez de la Torre Aaron Emmanuel

Este proyecto me permitió aplicar lo visto en la materia al construir un juego de UNO con interfaz gráfica en C#, conectado a una base de datos MySQL y a una API. En la primera etapa me encargué de la interfaz principal en Windows Forms, de mostrar las cartas de los tres jugadores como botones, de la base de datos, la conexión y los DAO (jugadores, historial y log del juego), del proyecto API con sus DTO y del doble búfer para evitar el parpadeo.

En la segunda etapa el juego pasó a una arquitectura cliente-servidor con sockets TCP, y me encargué de la parte del cliente: implementé ClienteSocket.cs, adapté InicioForms para conectarse al servidor, hice que UNO.cs usara el estado recibido por sockets y que AbanicoCartas mostrara únicamente la mano propia, envié las acciones de juego desde los controles de la interfaz e integré el servidor TCP con la API y MySQL. Definí con Alin el protocolo de comunicación y el envío de estados privados y públicos.

Lo más difícil fue lograr que el registro del log no congelara la interfaz y que las tres interfaces se mantuvieran sincronizadas sin mostrar las cartas de los demás jugadores. Resolverlo me enseñó a separar bien las capas del sistema y a pensar en el rendimiento desde el diseño. También aprendí a trabajar en equipo con Git y GitHub. Si lo repitiera, integraría la base de datos y la

interfaz desde etapas más tempranas para detectar errores antes. El resultado cumple con lo pedido: un juego funcional con historial de partidas y registro de cada movimiento.

Braga de Lima Brenda

Este proyecto me permitió aplicar mis conocimientos de programación orientada a objetos en C#, especialmente mediante la creación de clases para gestionar las cartas, el mazo, el reparto inicial y la pila de descarte del juego UNO. Uno de los principales retos fue conseguir que las diferentes funcionalidades se integraran correctamente con el resto del proyecto.

A medida que avanzaba el desarrollo, las tareas fueron adquiriendo un mayor nivel de complejidad, lo que exigió trabajar con más atención y cuidado, especialmente antes de realizar cada *commit*, para evitar errores y asegurar que los cambios no afectaran al funcionamiento de otras partes del proyecto.

Además, aprendí a organizar el desarrollo mediante *issues*, *branches* y *commits* en GitHub, comprendiendo la importancia de mantener el código organizado y trabajar de forma coordinada. Esta experiencia también me permitió comprender que el desarrollo en equipo requiere organización, adaptación y una buena comunicación para que las diferentes partes del proyecto puedan avanzar de manera conjunta.

En general, esta experiencia me ayudó a mejorar mis habilidades de programación y a comprender mejor cómo se desarrolla un proyecto de software en equipo.